

Scheduling Drone and Mobile Charger via Hybrid-Action Deep Reinforcement Learning

Jizhe Dou[✉], Haotian Zhang[✉], Yang Luo, and Guodong Sun[✉], *Member, IEEE*

Abstract—Recently, there has been a growing interest in using chargers to extend the operational longevity of UAVs (drones). In this paper, we explore a charger-assisted drone application where a drone observes points of interest while a mobile charger moves to recharge its battery. We focus on the route and charging schedule of the drone and mobile charger to maximize observation utility in the shortest possible time, while ensuring continuous drone operation. In our problem, the drone and mobile charger cooperate to complete a task. Their discrete-continuous hybrid actions pose a major computational challenge. To address this issue, we present a hybrid-action deep reinforcement learning framework, called HaDMC, which uses a typical policy learning algorithm to generate latent continuous actions. We specifically design and train an action decoder. It involves two pipelines to convert the latent continuous actions into the original hybrid actions for the drone and mobile charger to directly interact with environment. We incorporate a mutual learning scheme into model training, emphasizing collaboration over individual actions. By extensive numerical experiments, we evaluate HaDMC and compare it with state-of-the-art approaches. The experimental results demonstrate the effectiveness and efficiency of our solution.

Index Terms—Autonomous aerial vehicle, mobile charger, scheduling, reinforcement learning, hybrid actions.

I. INTRODUCTION

RECENT years have witnessed an unprecedented proliferation of autonomous aerial vehicles (commonly known as *drones*) in a wide range of applications for civilian operations, including environmental monitoring, search and rescue, traffic surveillance, aerial relays, and cartography [1], [2], [3]. The emergence of on-drone sensing and communication technologies makes it possible to gather data or information over large regions that are challenging or risky for human to access. Global commercial drone deployment is anticipated to grow as large as \$58.4 billion by 2026 [4]. Generally, small- and medium-sized commercial drones are powered by onboard batteries, and thus, the limited battery capacity is a major obstacle for drones to complete long-term tasks. Therefore, replenishing energy for drones

remains a major concern in drone-based sensing, networking, and trajectory planning.

The breakthrough development of recharging technologies presents a promising opportunity to prolong the lifespan of drones, attracting the attention of both academia and industry. Recently, a number of works have considered scenarios that involve one or more stationary charging stations, and the drones can fly to charging stations for energy replenishment. These works have focused on scheduling the drone's flights and charging to enhance system performance while avoiding battery depletion [5], [6]. In practice, however, stationary charging stations will incur high costs in terms of initial deployment and daily maintenance. In some scenarios, it may be challenging or even prohibited to build fixed-position charging stations.

In this paper, we explore the use of a mobile charger to recharge drone's battery, introducing a novel dimension to drone-based data collection: the integration of mobile charging and an emphasis on collaborative scheduling challenges. Specifically, the drone is required to fly over a set of points of interest (PoIs) to observe or collect data, while the charger can travel between designated charging points. When the drone is running out of energy, it can move to a nearby charging point along with the mobile charger. Both can then pause there to complete the battery recharging procedure without requiring human involvement. A motivating example of our scenario is collecting data from urban forests, in which there are sensor-equipped watchtowers and inspection paths constructed for fire trucks and visitors to access. The drone sequentially visits the watchtowers to gather data, while the mobile charger follows the inspection paths and pauses at suitable positions to recharge the drone. Another illustrative example is monitoring ecological dynamics on lake islands. The drone flies through the islands situated within a lake and observes each island for a duration, while the charger, installed on an autonomous boat, can dock at some near-shore locations to recharge the landing drone. From this general scenario, an important question naturally arises: *how to schedule the drone and mobile charger to achieve the maximum benefit from data collection in the shortest possible time, while ensuring that the drone never depletes its energy during data collection?*

It is very challenging to answer the above question. At first glance, this scheduling issue falls into the category of combinatorics. However, to obtain a drone-charger schedule, we must decide on the time for both PoI observation and drone charging, rather than simply selecting charging points for drone-charger rendezvous. Combinatorial approaches face challenges

Received 5 October 2024; revised 4 January 2025; accepted 11 March 2025. Date of publication 13 March 2025; date of current version 3 July 2025. This work was supported in part by the National Key Research & Development Program of China under Grant 2022YFF1302700. Recommended for acceptance by L. Kong. (Corresponding author: Guodong Sun.)

Jizhe Dou, Haotian Zhang, and Yang Luo are with the School of Information and AI, Beijing Forestry University, Beijing 100083, China.

Guodong Sun is with the School of Information and AI, Engineering Research Center for Forestry-Oriented Intelligent Information Processing, National Forestry and Grassland Administration, Beijing Forestry University, Beijing 100083, China (e-mail: sungd@bjfu.edu.cn).

Digital Object Identifier 10.1109/TMC.2025.3551386

in computational efficiency due to the involvement of continuous decisions on time. As a result, the lens of recent studies of drone's trajectory planning or scheduling has been on employing machine learning, particularly deep reinforcement learning, to find solutions in a data-driven way [7]. Unfortunately, most existing reinforcement learning methods cannot be directly applied to solving our problem because they only handle either discrete or continuous actions, whereas our scenario involves both discrete actions (deciding which PoI or charging point to visit) and continuous actions (determining the sojourn time at a PoI or charging point). Recently, a few reinforcement learning approaches, such as HPPO [8], MAHHQN [9] and HyAR [10], have been suggested for discrete-continuous hybrid action scenarios; however, they fall short in addressing our problem, primarily due to their inability to capture complex interdependencies inherent to the hybrid actions of the drone and mobile charger.

To address the above issues, we present HaDMC, a Hybrid-action reinforcement learning approach to the Drone and Mobile Charger scheduling, aimed at maximizing drone's observation efficiency. First, we propose a representation-learning methodology to convert our problem from a hybrid-action space into a continuous latent action space, allowing HaDMC to be efficiently trained in an off-policy and model-free way. Second, we design an action decoder, consisting of two separate pre-trainable modules, as the heart of our representation-learning methodology. Both modules can translate the continuous latent actions into original actions, by which the drone and mobile charger can directly interact with environment. Third, we introduce a semi-supervised pre-training method for the two modules of our action decoder, incorporating a mutual learning scheme. This enables our action decoder to develop the ability of learning action independencies between drone and mobile charger, emphasizing cooperative rather than individual actions. Finally, we design a structured reward function that is integrated into the HaDMC framework, effectively directing the training process. Our major contributions are summarized as follows.

- We present HaDMC, a discrete-continuous hybrid action reinforcement learning framework for the scheduling of drone and mobile charger, which is also adaptable for other applications based on hybrid-action agents.
- To address the challenge in learning the interdependency between drone and mobile charger, we propose a novel action decoder that decouples the decisions on discrete and continuous actions but can embed drone-charger cooperations in model training. This design principle also provides insight into broader multi-agent reinforcement learning problems involving interdependent hybrid actions.
- We conduct extensive numeric experiments to evaluate HaDMC and compare it with state-of-the-art models. The experimental results show the efficacy and efficiency of HaDMC in solving the proposed drone-charger scheduling problem.

The remainder of this paper is organized as follows: Section II presents a comprehensive review of the related literature. The scenario and the problem statement are elaborated in Section III. Section IV formalizes our problem within the

framework of a multi-stage reinforcement learning. The methodology, including the design and training algorithm for HaDMC, is thoroughly outlined in Section V. Experimental evaluation is provided in Section VI. Section VII offers a discussion of our findings and their implications. We draw a conclusion in Section VIII.

II. RELATED WORK

A. Drones for Data Collection

Due to the adaptable and mobile nature of drones, an increasing amount of research is dedicated to the drone control to effectively carry out data collection tasks by observing ground targets or gathering data from wireless sensors deployed on ground [11], [12], [13], [14], [15], [16], [17], [18], [19], [20]. Detailed and comprehensive reviews on drone-based data collection are available in [2], [21], [22].

In [11], an adaptive linear prediction algorithm is presented, which can generate a data transmission scheme to reduce energy consumption for data collection. Yuan et al. [12] propose a method of minimizing the completion time for data collection by joint user scheduling and drone trajectory design. Targeting the drone-aided data collection in large-scale IoT, Ma et al. [13] introduce an optimization algorithm to balance latency and energy cost by adaptively adjusting the IoT cluster size. Hu et al. [16] use a drone to collect data from IoT devices, aimed at minimizing the age of information (AoI) and drone's energy consumption. Li et al. [17] focus on planning the drone's trajectory to minimize AoI for data collection in wireless powered IoT systems. Ji et al. [20] consider the cellular networks with cached-enabled multiple drones, and use reinforcement learning to achieve optimal flight trajectories and communication performance. In [23], the multi-drone scheduling is investigated and a joint optimization in drone-enabled IoT scenarios is presented to accelerate task execution.

From these existing studies, it can be seen that an issue of major concern to researchers is to reduce the latency or improve the efficiency of data collection without violating the energy constraint of drones. The battery lifespans of commercial drones are usually tens of minutes [5], [24], [25]. For end-users who are interested in collecting data over expansive areas, there is a pressing requirement for drones with extended endurance capabilities.

B. Charger-Assisted Drone Scheduling

In the past few years, various recharging or replacement methods have been proposed for drones [5], including the use of wired or wireless power sources, as well as environmental energy (such as installing solar panels on drone).

Wireless Charging for Drones: Different from traditional wired or contact-based charging, wireless charging or power transfer for drones does not need cables or connection points and therefore, allows flexible charging alignment, quick connection, easy access, and even over-the-air charging [5], [6], [26]. In recent years, many commercial wireless charging stations or mobile charging platforms have been presented to extend the

battery lifespan of drones. For example, Powermat's technologies support 300-watt wireless charging for drones within 1.5 meters [27]. WiBotic designs and manufactures recharging solutions for drones [28], presenting a mobile landing pad which can wirelessly charge various drones in any weather. Warthog is an autonomous ground vehicle [29], which is suitable for all types of difficult terrains including steep areas and soft soils, and can move a payload of 272 Kg at a speed up to 5.3 m/s. If Warthog is equipped with a large-capacity battery, it can then be easily updated as a wireless mobile charger for drones. The advancement of wireless charging technology and autonomous robotics enables energy-limited drones to serve for longer, encouraging end-users to use wireless rechargeable drones for complex tasks that usually take a longer time to accomplish.

Drone Scheduling with Stationary Charger: In [30], a position-fixed wireless charging station is deployed to charge the drone, and the trajectory of drone is determined by a mixed integer linear programming model to achieve a minimal task latency. Similarly, Chen et al. [31] use a single charging station that emits resonant beams to charge a drone, and jointly optimize the drone's trajectory and the power efficiency of charging station. Chu et al. [32] collect data from ground sensors by a drone, which must fly back to the charging station before battery depletion, and present a deep learning-based solution for controlling flight speed and recharging process. In [33], the authors consider a grid-deployment scenario, with a fixed wireless charger in each grid, and train a Q-learning policy to charge a drone for collecting more data with less chargers. Fan et al. [34] consider the traffic monitoring scenario with multiple charging stations for drone charging, and propose a deep reinforcement learning approach, in combination with the attention mechanism, to determine drone's routing plan. Zhang et al. [35] optimize data transmission, energy consumption, and coverage fairness by optimizing the trajectory of a drone, which is powered by solar energy and charging stations. Li et al. [36] consider a multi-drone multi-charger scenario, schedule the chargers to turn on to charge near drones, and determine charging time; the authors aim to enhance the efficiency of chargers and model their problem as an optimization problem. The work in [37] also uses multiple stationary chargers to recharge crowdsensing drones, which are controlled by a reinforcement learning-based algorithm to obtain informative data collection.

Drone Scheduling with Mobile Charger: The authors in [25] use mobile chargers and propose a differential private framework of drone charging, which is integrated with a double auction-based charging schedule scheme. In [38], two drones are used to collaboratively collect data, with one drone wirelessly charging the other one, and multi-agent reinforcement learning is used to maximize the data throughput of ground IoT. Ribeiro et al. [39] use drones and mobile chargers to search in post-disaster areas, and assume that drones and chargers keep communication connectivity. They define a mixed-integer linear program model for a synchronized routing problem, employing a genetic algorithm to obtain an approximate solution. Liu et al. [40] use a drone to wirelessly charge the ground sensors, while employing a mobile vehicle (charger) to offer battery replacement for the drone; with a predetermined charger's route, the authors use a

deep Q-network to minimize the death time of sensors and the energy consumption of drone.

Remarks: Most of aforementioned works concentrate on finding drone's optimal trajectories that can improve data collection or charging efficiency. Their methods can fall into two categories: combinatorial and reinforcement learning-based approaches. Typically, optimizing trajectories of drones needs a large amount of computation and can even be computationally intractable. Therefore, combinatorial methods are suitable for small-scale, discrete, and certain scenarios where commercial solver or approximation algorithms can be deployed. In practice, the surrounding environment of drones and chargers is uncertain or time-varying and complex controls are needed, rendering combinatorial methods inapplicable. Reinforcement learning proposes a desirable alternative to hard combinatorial problems [41], as it can autonomously search heuristics by training an agent. The use of reinforcement learning to tackle optimization issues related to drone control or trajectory planning has become widely accepted as a predominant methodology.

C. Deep Reinforcement Learning

Reinforcement learning is a mathematical framework of developing autonomous agents that can interact with their environment based on experience and rewards. Recently, the potent amalgamation of reinforcement learning and deep learning has been playing a significant role in decision making, especially in the context of high-dimensional action space [42], [43]. Deep reinforcement learning has been applied in various domains such as robotics [44], healthcare [45], traffic engineering [46], [47], and more recently, in drone networking and controls [3].

Deep Reinforcement Learning with Discrete or Continuous Action: To address challenges associated with large spaces of state and action, DQN [48] uses neural networks to approximate Q-values, categorizing it under *value-based* reinforcement learning models. Furthermore, DQN employs an experience replay mechanism to achieve efficient off-policy training. Variants of DQN have been presented, such as Double DQN [49], Rainbow DQN [50], and NoisyNet [51], which are all value-based. Different from DQN-like models, TRPO (trust region policy optimization) [52] and PPO (proximal policy optimization) [53] are *policy-based* models, which can directly approximate the optimal policy with neural networks, while obtaining the probability distribution of each action [7]. The third category of deep reinforcement learning models, including DPG (deterministic policy gradient) [54], DDPG (deep deterministic policy gradient) [55], TD3 (twin delayed DDPG) [56], SAC (soft actor-critic) [57], and PPG (phasic policy gradient) [58], employs the *actor-critic* framework, which integrates value-based and policy-based learning. The critic part is learn Q-value, while the actor part is determine the policy. The actor-critic reinforcement learning is predominantly used in continuous-action scenarios.

Reinforcement Learning with Hybrid Action: Recently, a few studies have focused on effective controls over discrete-continuous hybrid actions. Based on PPO, Fan et al. [8] propose a hybrid actor-critic model (HPPPO), which uses multiple

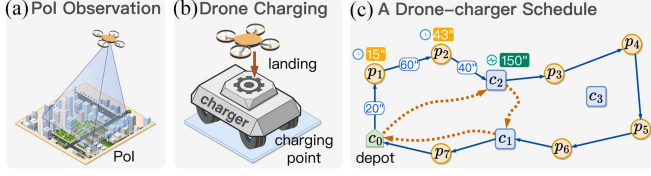


Fig. 1. Demonstration of our drone-charger scenario involving seven PoIs (marked as p_i) and three charging points (marked as c_j). The solid and dashed arrow lines represent the walks of the drone and the charger, respectively.

policy heads, with one for discrete action and others for continuous actions. In HPPO, the discrete and continuous action policies are trained as separate actors, which share a single critic. Different from HPPO, the PDQN model [59] and the Hybrid SAC model [60] consider the dependency of discrete and continuous actions; these models are essentially a hybrid structure, which uses a DQN and a DDPG to generate discrete and continuous actions, respectively. Li et al. [10] propose a hybrid-action representation architecture (HyAR) to learn a decodable continuous latent variable, from which original hybrid actions can be reconstructed. HyAR considers the possible underlying structure of hybrid-action space and improves learning scalability in comparison with previous models. However, it only considers the hybrid actions of an individual agent taking simple interactions with its environment. In [37], to optimize the data collection, the authors modify the loss function of PPO such that it can learn the combination of probability distributions of both discrete and continuous actions. The studies in [61] and [62] apply PDQN-like models for renewable building energy systems and data-center management, respectively. In multi-agent scenarios, the MAHHQN model [9] represents a typical approach in hybrid-action reinforcement learning. MAHHQN uses a mixing network to merge multiple agents into a single agent, and it first generates discrete actions and then, continuous ones. MAHHQN learns from original hybrid-action space, where the hybrid action spaces of all agents are homogeneous.

Remarks: Thus far, the majority of deep reinforcement learning models can only achieve either continuous or discrete control, but not both at the same time, precluding their direct applicability in our hybrid-action scenario. The proposed approaches for hybrid-action reinforcement learning do not take into account the complex dependency among discrete and continuous actions, nor do they focus on the collaboration of multiple agents that all take hybrid actions. Besides, those learning approaches for hybrid actions have distinct scenarios and objectives that are not aligned with ours, and therefore, are unsuitable for addressing our specific challenges.

III. SYSTEM MODEL AND PROBLEM STATEMENT

A. System Model and Assumptions

We consider a monitoring area that involves a set of PoIs, denoted as $P = \{p_1, p_2, \dots, p_n\}$. A drone is used to collect observational data from all PoIs by sequentially flying to and hovering over them, as depicted in Fig. 1(a). Due to limited energy capacity, the drone cannot collect data from all PoIs in a single flying tour, and thus, a mobile charger that carries sufficient

energy is used to recharge the drone. As shown in Fig. 1(b), the mobile charger is equipped with a charging platform for the drone to land on. Also, there is a set C of *charging points*, typically located in open areas to ensure easy access for the charger and safe landing for the drone. Only at charging points can the drone meet the mobile charger and land on its charging pad for energy replenishment. Initially, both drone and mobile charger are located at a *depot*, which is stationary and used to maintain the drone and mobile charger. The depot, denoted by c_0 , can also be used as a charging point.

In our scenario, the drone is required to visit all PoIs, and the visit sequence is determined in advance. For ease of expression, we assume that $p_{i+1} \in P$ is the next PoI to be visited after $p_i \in P$. Fig. 1(c) exemplifies our scenario. After setting off the depot, the drone takes 20 seconds to fly to p_1 , staying there for 15 seconds for observation. Then, it flies to p_2 and conducts a 43-second observation. Leaving p_2 , the drone flies to charging point c_2 and stays there for a period of 150 seconds, which includes the recharging time and the possible wait for the mobile charger to arrive.

As demonstrated in Fig. 1(c), the entire task can be represented by two weighted closed walks, traversed by the drone and charger, respectively. The drone's walk starts and ends both at the depot, traveling over all PoIs and some charging points. Similarly, the charger's walk also starts and ends at the depot, but only passing across charging points. These two walks form a *drone-charger schedule*, denoted by E , which involves not only the walks but also the sojourn time at PoIs and the charging time at charging points. We use E_i to represent part of schedule E , in which only the first i PoIs of P have been observed by the drone. We use $t(E)$ to denote the total time spent by the drone in flight and recharging during the schedule E . Clearly, $t(E)$ is the time cost associated with observing all PoIs.

B. Utility Model and Problem Statement

Intuitively, the longer a drone's sojourn above a PoI is, the richer information it observes. We use $\tau_i^{\min}$ and $\tau_i^{\max}$ to represent the minimum and maximum time requirement for the drone to observe PoI p_i , respectively. In other words, observations less than $\tau_i^{\min}$ cannot obtain any effective information, while no additional information can be obtained after the observation time exceeds $\tau_i^{\max}$. If the drone observes p_i for τ_i time, then the *utility of observation* obtained by the drone at p_i is calculated by

$$\nu_i(\tau_i) = \begin{cases} 0 & \text{if } \tau_i < \tau_i^{\min} \\ \min\{\tau_i/\tau_i^{\max}, 1\} & \text{otherwise.} \end{cases} \quad (1)$$

PoIs may vary in their importance. For a more important PoI, we need to spend more time in observing it. Based on this, the *importance of PoI* p_i is measured by

$$\zeta_i = \tau_i^{\max} / \sum_{p_j \in P} \tau_j^{\max}. \quad (2)$$

Combining (1) and (2), the total utility of observation obtained by the drone through the schedule E can be calculated as $u(E) = \sum_{p_i \in P} [\zeta_i \cdot \nu_i(\tau_i)]$. Note that other practical definitions of observation utility and PoI importance can also be applied to

TABLE I
MAIN NOTATIONS IN USE

notation	description
P	$\{p_i 1 \leq i \leq n\}$, the sequence of PoIs that must be visited sequentially by drone.
C	$\{c_i 0 \leq i < m\}$, the set of charging points, where c_0 represents the depot.
e	the energy capacity of the drone
e_j	the remaining energy of drone when it has just arrived at $c_j \in C$
γ_f	drone's energy consumption rate during flight
γ_o	drone's energy consumption rate during observation
γ_c	charger's recharging rate
τ_i	the time for observing $p_i \in P$
$[\tau_i^{\min}, \tau_i^{\max}]$	the range for τ_i ($0 < \tau_i^{\min} \leq \tau_i^{\max}$)
$\tilde{\tau}_j$	the time for charging the drone at $c_j \in C$, and $\tilde{\tau}_j \leq (e - e_j)/\gamma_c$
$t(x, y)$	the time for drone's flight from x to y ($x, y \in P \cup C$)
$\tilde{t}(x, y)$	the time for charger's movement from x to y ($x, y \in P \cup C$)
P_i	a subset of P , which is formed by the first i PoIs of P ($0 \leq i \leq n$)
E_i	part of the drone-charger schedule, in which only PoIs of P_i have been observed

our approach, as long as they are non-decreasing functions with respect to the observation time. Table I lists the main notations related to our system deployment.

Besides observational utility, data timeliness is also a key concern for end-users in many practical applications, especially in delay-sensitive applications. In this paper, therefore, we aim to find a drone-charger schedule E that can achieve high utility of observation with a low time cost. This problem can be expressed with

$$\max_E : u(E)/t(E), \quad (3)$$

which is constrained by the requirement for the drone to monitor all PoIs without depleting its battery before recharging or returning to the depot.

We consider a simplified instance of our problem: the drone observes each PoI for an equal duration, and the charger can move between charging points instantaneously, incurring no time cost. That is, we only need to decide on which charging points to select and the duration for drone recharging to obtain a time-shortest schedule. It is easy to know that this particular class of problem instance is fundamentally a mixed integer programming problem, which has proven NP-hard in general contexts. Although our problem involves a finite number of decision-making stages, solving it with traditional reinforcement learning is prohibitively time-consuming due to the large number

of possible actions for the drone and charger. The curse of dimensionality motivates new optimization approaches that strike a balance between computation complexity and performance. Another challenge facing our problem is that it involves discrete-continuous hybrid actions. The nature of making hybrid actions on multiple agents hinders most of existing reinforcement learning approaches, which are typically proposed for either discrete or continuous scenarios.

IV. DECISION CONTROL-BASED FORMULATION

Due to the challenge of our problem, we first formulate it as a multi-stage decision process, denoted by $\langle \mathcal{S}, \mathcal{A}, r, \gamma \rangle$, and then propose an approach based on hybrid-action reinforcement learning. This multi-stage decision process will terminate when the drone completes its task and returns to the depot. Here, $\mathcal{S}$ is the joint state space of drone and charger, and $\mathcal{A}$ is their joint action space. Function $r : \mathcal{S} \times \mathcal{A} \rightarrow \mathbb{R}$ is the reward function, which measures the reward for current stage if action $a \in \mathcal{A}$ is executed in state $s \in \mathcal{S}$. Parameter γ is the discount factor in interval of $(0, 1]$. For given s and a , the transition to next state is typically probabilistic. The objective is to maximize the value of $\sum_{i=1}^N \gamma^{i-1} r_i$ when the decision process terminates with N stages. This value represents the expected return or cumulative reward. We will next detail $\langle \mathcal{S}, \mathcal{A}, r, \gamma \rangle$ in reinforcement learning terminology.

A. State Space

The state space of our system can be characterized with a set $\mathcal{S}$, in which each element is a tuple that puts together the states of the drone, the charger, PoIs and charging points.

State of the drone: a vector of parameters for the drone, including its position, velocity in flight, energy consumption rates for both flight and observation, and current remaining energy.

State of the mobile charger: a vector of mobile charger's states, including its position, velocity in movement, and charging rate.

State of PoIs: a vector of parameters associated with all the PoIs, in which the state of each PoI includes its position, range of observation time, and the assigned observation time. If a PoI has not been visited yet, its observation time is set to zero.

State of charging points: the vector of all charging points' positions. Each charging point c is associated with a time value. If the drone and mobile charger do not meet at c , we associate c with zero; otherwise, with the actual duration of charging process. We put depot's position into this vector because the depot can also be thought of as a particular charging point.

B. Action Space

At the beginning of the k th stage, if the first unobserved PoI is p_i , the drone needs to make a decision or take an action: either flying from current position to p_i and conducting an observation of τ_i time, or flying to meet the charger at a specific charging point for energy replenishment. The drone's action space for the k th stage is denoted by $A_k = \{(a_k, \tau_i)\}$, where $a_k \in \{0, 1\}$

while τ_i is equal to 0 if $a_k = 0$, or to a specific value within $[\tau_i^{\min}, \tau_i^{\max}]$ if $a_k = 1$. For example, if an action made by the drone is (1, 25.6), the drone will fly to the next PoI and conduct an observation for 25.6 time units. In contrast, the action of (0, 0) will direct the drone to fly towards a charging point determined by the charger. Apparently, the drone makes binary (discrete) decisions about the flight to subsequent PoI, and continuous decisions about the time length of its observation.

When the drone is making a decision in stage k , the charger must also determine whether to remain at its current position or move to another charging point to recharge the drone. We use $\tilde{A}_k = \{(\tilde{a}_k, \tilde{\tau}_k)\}$ to denote charger's action space for the k th stage. Here, $\tilde{a}_k$ is a charging point, say $c_j \in C$, that the charger can stay at or move to, while $\tilde{\tau}_k$ is the time spent in recharging if the drone and the charger meet at c_j . Noticeably, if the drone is flying from a PoI for energy replenishment, it may have to land on a charging point that is close to this PoI because of limited residual energy. Denote by $C_k \subseteq C$ the set of charging points that the drone can reach in the k th stage. Consequently, we must have $\tilde{a}_k \in C_k$, which shrinks the action space to search for each stage during the process of model training. Similar to the drone's action space, the charger's action space is also hybrid: the movement action is discrete and the time for recharging the drone is continuous. In our scenario, the drone and mobile charger play different roles in different action spaces (i.e., they are heterogeneous), yet their actions or their interplay must be coordinated for effective learning policies. For example, when the drone decides to fly to observe the next PoI, the mobile charger's action of moving to charge the drone becomes irrelevant and invalid. Learning the interdependency of the drone and the mobile charger is essential for effectively addressing our problem.

C. Reward Function Design

Intuitively, we could directly use the objective function defined in (3) to measure the reward acquired at the end of each stage. However, the evaluation of objective value requires obtaining $t(E)$ in advance, which is impossible unless the entire task is finished. To address this contradiction, we design a reward function that can be evaluated in each stage based only on the actions and state transition that have already happened in the previous stage.

To articulate the design principles of our reward function, we consider the very beginning of a stage k ($k \geq 1$), which is profiled as follows. First, PoIs of P_i have been already observed, i.e., the subsequent PoI to be observed is p_{i+1} , where $0 \leq i \leq n$. Note here that p_0 and p_{n+1} are not included in P , but both are specifically equivalent to c_0 (i.e., the depot) and then, τ_0 and τ_{n+1} can reasonably be set to zero. Second, the drone stays at a position $x \in \{p_i, c_j\}$, having completed its observation task at p_i or finished the charging process at $c_j \in C$, while the charger is at charging point c_j . Third, the drone's remaining energy level is e_x and it can perform any possible action of A_k as long as it has enough energy, while the charger can perform any action of $\tilde{A}_k$. As shown in Fig. 2, our reward function is structured to exhibit four different forms under four joint-action cases.

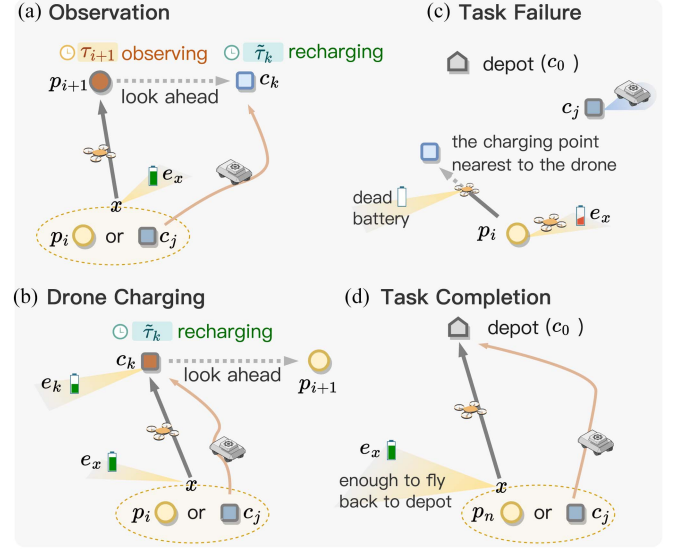


Fig. 2. Four cases where the rewards are calculated based on the specific states and actions.

1) *Case A: Reward for Observation:* In this case, as shown in Fig. 2(a), the drone currently stays at x , which is PoI p_i or the charging point c_j (where the charger remains). If the drone flies to observe p_{i+1} for τ_{i+1} time while keeping within the energy constraint, it will acquire a reward of

$$r^{\text{obs}} = \frac{(i+1) \cdot u(E_{i+1})}{t(E_{i+1})} \times \left(\frac{\tau_{i+1}}{t(x, p_{i+1})} + \xi_1 \right), \quad (4)$$

where two major terms are multiplied, E_{i+1} is the part of schedule E that will be completed by the end of current stage, and ξ_1 is a non-negative variable that depends on the actions of both the drone and the charger. In the first term, $u(E_{i+1})/t(E_{i+1})$ measures the observation efficiency up to the conclusion of current stage. It is easy to understand that this term is incentivize the drone to continue its flight to the next PoI. Such an efficiency-based incentive should be amplified with more and more PoIs observed, i.e., with i increasing. So we introduce $(i+1)$ to the first term as a multiplier. In the second term, $\tau_{i+1}/t(x, p_{i+1})$ indicates that actions with a short flight time but a long observation time can lead to higher rewards. Besides, we use ξ_1 as an additional incentive for the drone to explore the following PoI if there is no danger of energy depletion. The value of ξ_1 is determined by the following expressions.

$$\Delta t_1 = \max\{1, t(x, p_{i+1}) + t(p_{i+1}, c_k)\} \quad (5)$$

$$\Delta e_1 = \gamma_f \cdot \Delta t_1 \quad (6)$$

$$\Delta e'_1 = \Delta e_1 + \gamma_o \cdot \tau_{i+1}^{\max} \quad (7)$$

$$\xi_1 = \begin{cases} 1/\Delta t_1 & \text{if } \Delta e'_1 \leq e_x \text{ and } \Delta e_1 \leq e/2 \\ 0 & \text{otherwise} \end{cases} \quad (8a)$$

$$(8b)$$

Suppose that in current stage, as shown in Fig. 2(a), the charger decides to move from c_j to c_k . Note that c_k can be equivalent to c_j , i.e., the charger remains at c_j in current stage. In (5), Δt_1 calculates the drone's total flight time in its current and next stage, if it flies to c_k in the next stage to meet the charger. In the event that Δt_1 assumes a duration shorter than one

time unit (although this situation is actually very unlikely to occur), it will be adjusted to one to assure $\xi_1 \leq 1$. In (6), Δe_1 represents the drone's energy in flight, and $\Delta e'_1$ represents the maximum possible energy consumed by the drone in both flight and observation at p_{i+1} . It is worth noting that the evaluation of (5), (6), and (7) relies on the calculation of $t(p_{i+1}, c_k)$, by which the drone looks ahead to possible subsequent scenarios before making decisions. Specifically, if the condition in (8a) is met, the drone's current energy e_x is adequate for the following stage (in which the drone flies to meet the charger at c_k for energy replenishment), and then ξ_1 is set to $1/\Delta t_1$, i.e., giving the drone an additional incentive. This lookahead or forward-thinking policy encourages the drone to explore in current stage while also considering potential charging opportunities in the subsequent stage.

2) *Case B: Reward for Drone Charging:* Fig. 2(b) depicts a case, where the drone departs from x (p_i or c_j), heading for a charging point $c_k \in C_k$. If $0 < i < n$, this scenario can possibly arise after the drone finishes its observation at p_i or is recharged by the charger at c_j . The corresponding reward is calculated by

$$r^{\text{chg}} = \begin{cases} 0 & \text{if } \Delta e_2 \geq \xi_2 \cdot e_x \\ \frac{i \cdot u(E_i)}{t(E_i)} \times \frac{e}{e_k} \times \frac{\tilde{\tau}_k}{\Delta t_2} & \text{otherwise,} \end{cases} \quad (9a)$$

$$(9b)$$

where e_x and e_k are remaining energy levels of the drone when it departs from x and arrives at c_k , respectively, ξ_2 is within (0, 1), and Δe_2 and Δt_2 are defined below.

$$\Delta e_2 = \gamma_f \cdot t(x, c_k) \quad (10)$$

$$\Delta t_2 = \max\{1, \max\{t(x, c_k), \tilde{t}(c_j, c_k)\}\} + t(c_k, p_{i+1}) \quad (11)$$

In (10), Δe_2 measures the energy consumed by the drone flying from x to c_k . In (11), the term $\max\{t(x, c_k), \tilde{t}(c_j, c_k)\}$ measures the time required for the drone or the charger to meet each other at c_k . Therefore, like Δt_1 in (5), Δt_2 can calculate the total time for the drone to travel from current position x to the subsequent unobserved PoI p_{i+1} .

In (9a), we set a threshold for Δe_2 . A large value of Δe_2 indicates that the charging point c_k is relatively far away from current position x . Therefore, the charger will likely take longer to reach c_k , resulting in increased latency. We neither encourage nor discourage such an action. In (9b), the efficiency-based incentive (the first term) is also used. Besides, we use the second term to encourage a drone-charger rendezvous if drone's battery is low. Using the lookahead policy, the third term $\tilde{\tau}_k/\Delta t_2$ considers p_{i+1} and encourages both drone and charger to select a rendezvous relatively close to p_{i+1} and c_j , thereby resulting in drone's expeditious arrival at p_{i+1} in the next stage.

3) *Case C: Reward or Penalty for Task Failure:* There is possibly a case as shown in Fig. 2(c): after observing p_i , the drone lacks enough energy to reach the next PoI or any charging points, including the depot. In other words, the drone's battery will be depleted on flight if it takes off from p_i . This case represents a failure of current drone-charger schedule, necessitating the imposition of a penalty or a negative reward. This penalty is

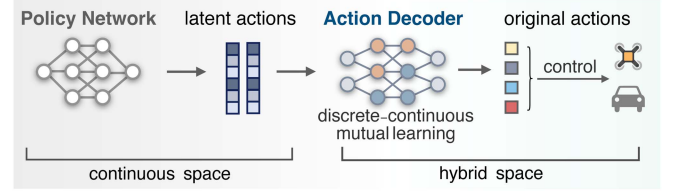


Fig. 3. Basic idea of the proposed HaDMC approach.

expressed with

$$r^{\text{fail}} = \xi_3 \cdot \left(n - \sum_{1 \leq l \leq i} \nu_l(\tau_l) \right), \quad (12)$$

where the penalty parameter ξ_3 is negative and $\nu_l(\tau_l)$, defined in (1), is the observation utility obtained at PoI p_l . If this scenario arises with a small value of i , it suggests that the current task should have encountered an early failure, and a significant penalty needs to be incurred. It is clear that r^{fail} is always nonpositive.

4) *Case D: Reward for Task Completion:* When the drone finishes its observation at p_n , the last PoI of P , we encourage it to fly directly back to the depot if it has sufficient remaining energy to do so. This case is depicted in Fig. 2(d), and the corresponding reward is expressed with

$$r^{\text{end}} = \xi_4 \cdot \frac{u(E_n)}{t(E_n) + t(p_n, c_0)}, \quad (13)$$

where ξ_4 is a positive scalar. In this scenario, the charger also returns to the depot, which is independent of the drone's actions. Therefore, we only consider drone's action when determining the reward for completing the entire task.

V. MODEL DESIGN

A. Overview

The critical challenge of applying reinforcement learning to our system is to learn an effective hybrid-action policy, by which both drone and mobile charger can take cooperative actions to optimize observation efficiency. To address this hybrid-action issue, we propose HaDMC and its basic idea is illustrated in Fig. 3.

Motivated by the representation learning paradigm, we introduce a representation methodology for the hybrid-action space of the drone and charger, and use conventional policy learning models to generate latent continuous actions. These latent actions cannot support the drone and charger to directly interact with environment. To make them meaningful or understandable for both devices, we specifically design and train an action decoder to convert the latent actions into original actions. Based on the original actions and the corresponding reward scenarios, the drone and charger can interact with environment, while updating system states and propelling the system forward. In the action decoder, we employ two separate pipelines to generate discrete and continuous actions, respectively. Besides, we facilitate the mutual learning between the two pipelines, by directing their outputs forward each other as input during the process of model training. Through the mutual learning, the action decoder can

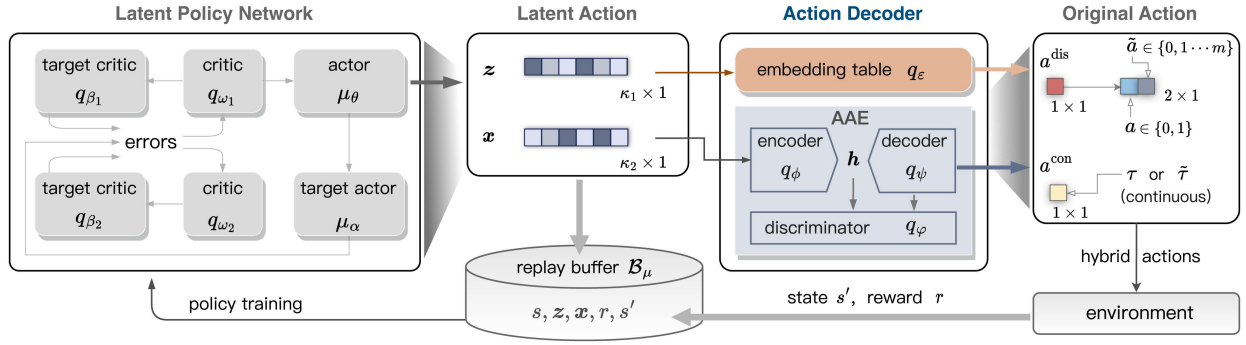


Fig. 4. Overall architecture of the learning model of HaDMC.

develop the ability to generate a joint action for both drone and charger, emphasizing their collaborative rather than individual actions. In summary, HaDMC first makes latent decision in a continuous space and then derives original actions in hybrid spaces.

B. Architecture of HaDMC

The overall architecture of HaDMC and its training framework are shown in Fig. 4. In HaDMC, the latent policy network follows the actor-critic reinforcement learning, which renders HaDMC off-policy, i.e., a replay buffer can be used to facilitate model training. In the implementation of HaDMC, we employ TD3 [56], the most popular policy-learning model, to generate latent actions. Actually, any actor-critic models for continuous action can serve as the latent policy network. The reason for the preference of the actor-critic structure in HaDMC is as follows. In practice, reinforcement learning can be implemented by value learning or policy learning. Value learning is suitable for finite or discrete action spaces. Policy learning, exemplified by REINFORCE [63] and actor-critic, is suitable for continuous action spaces. REINFORCE often results in high variance and noise gradients because of the huge difference among action trajectories. By contrast, the actor-critic structure can output continuous actions or their distribution from its *actor* part and then, evaluate action values at its *critic* part. Critic improves itself based on actor's interaction with environment, while actor updates its policy according to critic's evaluation and interacts with environment using new policy. In this way, the actor-critic structure can make a balance between value learning and policy learning. Recently, several actor-critic policy networks have been proposed for continuous-action scenarios, including TD3 and DDPG, and have gained widespread acceptance as a fundamental framework in the field of reinforcement learning.

The policy network of HaDMC generates two continuous latent vectors, z and x ; their sizes are κ_1 and κ_2 , respectively. All elements of both latent vectors are within $[-1, 1]$. The crucial part of HaDMC is to learn how to derive original actions from the two latent actions. We design an action decoder, which comprises of two modules or pipelines: an embedding table q_ϵ and a modified adversarial autoencoder (AAE). The embedding table maps latent action z to a discrete scalar a^{dis} , while the AAE module maps latent action x to a continuous scalar a^{con} .

There are several kinds of widely-used autoencoder models, such as AutoEncoder [64], VAE (variational autoencoder) [65], and AAE (adversarial autoencoder) [66]. AAE is more powerful than other types of autoencoder because it can effectively learn about unknown distribution. This ability comes with an adversarial component that discriminates the unknown distribution with a designated distribution (such as Gaussian distribution). Therefore, in our action decoder, we select AAE, with a minor adjustment made to output channels. Our action decoder outputs two scalars: a discrete scalar a^{dis} and a continuous scalar a^{con} . HaDMC involves a method that can extract all original discrete and continuous actions solely from these two scalars.

C. Representation of Hybrid Actions

Recently, a few studies have implemented reinforcement learning in hybrid-action spaces. One common approach is use Gaussian distribution to approximate the distribution of continuous actions [37], [67]. For example, the J-PPO model [37] addresses the hybrid-action issue by applying Gaussian approximation to continuous action and treating the distribution of hybrid actions separately. In practice, action spaces are usually bounded, which means that certain actions generated by the Gaussian may fall outside valid action spaces, leading to estimation errors [68], [69]. In addition, the probability distribution of continuous actions may be not Gaussian. HyAR [10] is a reinforcement learning framework proposed only recently for the single-agent hybrid-action scenario. It generates continuous actions in a latent layer that represents the interconnection between discrete and continuous actions, and uses a conditional variational autoencoder to derive original actions. However, HyAR's latent actions are also limited to a Gaussian distribution, similar to J-PPO and HPPO. During model training, HyAR uses a predictor to predict subsequent states and compare them with actual states, to improve the representation ability of hybrid action. However, this ability improvement is contingent upon dynamics of system states. In our scenario, the system states do not vary significantly in two consecutive stages, irrespective of what actions are performed by the drone and charger. This poses a significant challenge in designing an effective hybrid-action learning method that can discern nuanced dynamics.

In the HaDMC design, we propose a novel representation learning-based approach (i.e., the action decoder in Fig. 4), to

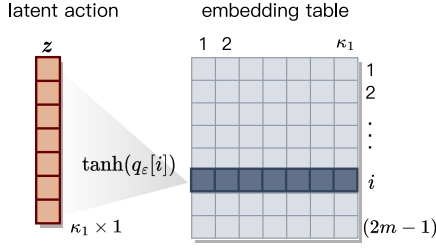


Fig. 5. Demonstration of our embedding table in converting latent continuous action $\mathbf{z}$ to discrete action a^{dis} .

translate latent continuous actions into original hybrid actions. Before delving into the specifics of our action decoder, we will first introduce how to handle the two distinct discrete actions of the drone and charger at the same time.

1) *Combining Drone's and Charger's Discrete Actions:* At the beginning of the k th stage, the drone makes a discrete action a_k : flying either to observe the next PoI or to meet the charger at a specific charging point. Meanwhile, the charger also makes a discrete decision $\tilde{a}_k$, on its movement. We combine the two separate discrete actions such that only a single latent policy model is necessary. For ease of expression, we will omit the stage sequence k when it comes to actions in later sections. The approach to combining the two discrete actions is formally expressed with

$$a^{\text{dis}} = m \cdot a + \tilde{a}, \quad (14)$$

where m is the number of charging points. Since $a \in \{0, 1\}$ and $\tilde{a} \in C$ with $|C| < m$, multiplying a by a positive integer can strengthen the effect of a on the value of a^{dis} if $a = 1$. We always have $0 \leq \tilde{a} \leq a^{\text{dis}} \leq 2m - 1$. On the other hand, once a^{dis} is given by the action decoder of Fig. 4, we can decompose the values of a and $\tilde{a}$ from a^{dis} , simply by a division operation. That said, when a^{dis} is divided by m , the resulted quotient and remainder are a and $\tilde{a}$, respectively.

2) *Mapping Latent Action to Original Action:* In our action decoder, the hybrid-action representation learning separates into two parallel pipelines: mapping the latent actions $\mathbf{z}$ and $\mathbf{x}$ (both selected by the latent policy network) to a^{dis} and a^{con} , respectively.

For the first mapping task, we leverage an embedding table q_ε with learnable parameter ε , which is pre-trained and can convert $\mathbf{z}$ to a^{dis} . For any $1 \leq i \leq 2m - 1$, the i th row of q_ε , denoted by $q_\varepsilon[i]$, is a continuous vector of size $\kappa_1 \times 1$. As illustrated in Fig. 5, specifically, such a conversion is formulated as

$$a^{\text{dis}} = \arg \min_i d(\mathbf{z}, \tanh(q_\varepsilon[i])), \quad (15)$$

where function $d(\cdot, \cdot)$ calculates the Euclidean distance between the two input vectors, and the $\tanh$ function is used to normalize $q_\varepsilon[i]$ such that any elements of $q_\varepsilon[i]$ are within the range of $[-1, 1]$.

To map $\mathbf{x}$ to an original action a^{con} , which represents observation time τ or charging time $\tilde{\tau}$, we deliberately train an adversarial autoencoder (AAE). Typically, an AAE model involves three major components: an *encoder* q_ϕ , a *decoder* q_ψ , and a *discriminator* q_φ ; they are usually neural networks with ϕ , ψ and φ as learnable parameters. Essentially, AAE is a generative

autoencoder, in which the encoder q_ϕ maps input data to a latent vector $\mathbf{h}$ (an informative representation of the input), and then the decoder q_ψ reconstructs the original input from $\mathbf{h}$. Different from standard autoencoder, however, AAE fuses with the concept of generative adversarial network; specifically, it integrates a discriminator q_φ that is responsible for distinguishing between real and fake (or generated) latent samples. Additionally, AAE employs an adversarial training approach: training the encoder to generate realistic latent samples to confuse the discriminator, while training the discriminator to gradually enhance its ability to distinguish the real from the generated samples. Such an adversarial training process enables our action decoder to effectively capture representative and significant features within the latent space.

In our action decoder, the inference of the AAE module is formally equivalent to $a^{\text{con}} = q_\psi(\mathbf{h})$, where $\mathbf{h} = q_\phi(\mathbf{x})$. The meaning of a^{con} depends on values of the discrete output a^{dis} . According to (14), if a^{dis} can lead to $a = 1$, then the value of a^{con} represents the time of the drone observing the subsequent PoI. If $a = 0$ and $\tilde{a} \neq 0$ are derived from a^{dis} , then the value of a^{con} represents the time spent by the charger in recharging the drone at charging point $\tilde{a}$.

3) *Summary:* The HaDMC framework introduces a novel action decoder based on representation learning to generate original hybrid actions from a latent continuous space. This action decoder separates yet interlinks the learning processes of discrete and continuous actions, ensuring that the essential interdependencies between the drone's and mobile charger's hybrid actions are preserved. Our action decoder can be naturally extended beyond the two-agent case. For instance, HaDMC can be scaled to larger multi-agent systems by replicating the fundamental pipelines, while still accommodating the mutual learning scheme proposed in Section V-D2, which fosters interdependence learning across hybrid actions. Additionally, without altering the action decoder's architecture, it can be adapted to multi-agent cases, by combining the discrete actions of all agents into a single, decomposable representation and enabling the AAE module to output the continuous actions for all agents simultaneously in a vector. These extensions, though requiring certain refinements, underscore that our design principles remain relevant and transferable to multi-agent settings involving interdependent hybrid actions.

D. Learning Algorithm for HaDMC

HaDMC is trained by Algorithm 1, which involves three primary phases: initializing the entire model, pre-training the action decoder, and training the latent policy network.

1) *Initialization of Model:* Before training HaDMC, we use the Kaiming Initialization method [70] to initialize all parameters within the policy network and AAE module of HaDMC. This initialization method is chosen due to its consideration of the nonlinearity of activation functions and its extensive use in training neural networks. We initialize the embedding table by using a zero-centered Gaussian distribution with a standard deviation of one, and clip the embedding table's parameters to the range of $[-1, 1]$ by value.

Algorithm 1: Training the HaDMC Model.

- input :** parameters related to system deployment (such as P, C, e , etc.) as well as other parameters used in training (such as learning rate η , discount ratio λ , etc.)
- Output:** a trained HaDMC model, which can generate a drone-charger schedule
- ▷ **initializing the HaDMC model;**
 - 1: Initialize all learnable parameters of our model ;
 - 2: Establish the replay buffer $\mathcal{B}_\pi$ with a random policy of action
 - ▷ **training HaDMC's action decoder;**
 - 3: **while** step $i = 1, 2$ up to n_π
 - 4: Randomly select a batch of b_π tuples from $\mathcal{B}_\pi$;
 - 5: Calculate the loss values of L_1, L_2 and L_3 ;
 - 6: Update the parameters of the action decoder, including $\varepsilon, \phi, \varphi$ and ψ
 - ▷ **training HaDMC's latent policy network;**
 - 7: Use the initialized policy network to prepare b_μ tuples and store them in $\mathcal{B}_\mu$, preparing for subsequent model training
 - 8: **while** step $i = 1, 2$ up to n_μ
 - 9: Make the latent policy network μ_θ generate the latent actions z and x , both with an exploring noise $\epsilon \sim N(0, \sigma)$ added on each dimension of z and x
 - 10: Feed z into the embedding table q_ε of action decoder and output a^{dis} , from which the drone's and charger's discrete actions (i.e., a and $\tilde{a}$) can be determined by (14).
 - 11: Feed x into the AAE module to obtain a^{con} , the time for PoI observation or for drone charging
 - 12: Make the original actions obtained above interact with the environment, transitioning the system state from s to s'
 - 13: Calculate the reward r based on the current scenario, and put the tuple $\langle s, z, x, r, s' \rangle$ into the experience replay buffer $\mathcal{B}_\mu$
 - 14: Update the latent policy network with a random mini-batch of b_μ tuples selected from $\mathcal{B}_\mu$, during which, a clipped policy noise $\epsilon' \sim N(0, \sigma')$ is used for the target actor to output actions
 - 15: **return**

The action decoder is map the latent actions selected by the latent policy network back to the original hybrid actions. Before training the entire model, we pre-train the action decoder with an experience replay buffer $\mathcal{B}_\pi$ established in advance. The experiences or tuples in $\mathcal{B}_\pi$ are all generated by an independent simple policy model that randomly selects actions from the joint action space $\mathcal{A}$ and interacts with environment. More specifically, we first take random actions $(a^{\text{dis}}, a^{\text{con}})$ from a uniform distribution, and then we obtain a tuple $\langle s, a^{\text{dis}}, a^{\text{con}}, r, s' \rangle$, according to the interaction with environment, meanwhile putting it into $\mathcal{B}_\pi$. This process of selecting actions continues until $\mathcal{B}_\pi$ reaches its maximum capacity. During establishing $\mathcal{B}_\pi$, the use of a random policy for uniformly selecting actions aims to collect unbiased and diverse experiences in a model-free way.

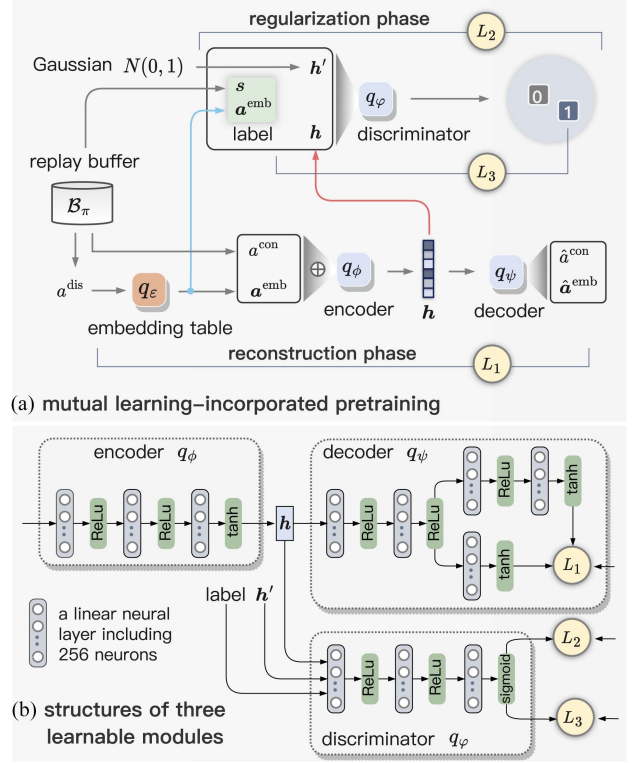


Fig. 6. Pre-training process of HaDMC's action decoder.

2) *Pre-Training the Action Decoder:* The process of pre-training our action decoder is illustrated in Fig. 6(a), and the AAE module, with modifications made for adaption to the pre-training, is depicted in Fig. 6(b). We iteratively select a random mini-batch of b_π tuples from $\mathcal{B}_\pi$ to train the embedding table q_ε and AAE module. This procedure is iterated n_π times. During the action decoder pre-training, a critical issue is to ensure that the interdependencies among all hybrid actions can be effectively learned. For example, if the discrete output from the embedding table directs the drone to charge, then the continuous output from the AAE should be accordingly interpreted as the charging time. To address this issue, we introduce a mutual learning policy to the pre-training process: the embedding table updates its parameters based on the AAE's output, while the AAE, in turn, learns from the embedding table's output. This process is detailed as follows.

Consider a tuple $\langle s, a^{\text{dis}}, a^{\text{con}}, r, s' \rangle$ selected from $\mathcal{B}_\pi$. We feed a^{dis} to the embedding table, which outputs a continuous vector $a^{\text{emb}} = q_\varepsilon(a^{\text{dis}})$. Then, the concatenation of a^{emb} and a^{con} is input to the AAE module. Along the forward path of AAE, the encoder q_ϕ first encodes this concatenation result into h , a hidden vector of $\kappa_2 \times 1$, and then, the decoder q_ψ decodes or reconstructs h into the vector $(\hat{a}^{\text{emb}}, \hat{a}^{\text{con}})$. Typically, training AAE includes two phases: reconstruction and regularization. In the reconstruction phase, both encoder q_ϕ and decoder q_ψ are updated through minimizing loss L_1 that is defined by

$$L_1 = \alpha_1 \cdot f_M(\hat{a}^{\text{emb}}, a^{\text{emb}}) + (1 - \alpha_1) \cdot f_M(\hat{a}^{\text{con}}, a^{\text{con}}), \quad (16)$$

where function f_M calculates the mean squared error of the two inputs and α_1 is a parameter within $(0, 1)$. In addition to the parameters of AAE's encoder and decoder, we also update the parameters of the embedding table q_e to reduce L_1 , enabling q_e to acquire knowledge from AAE's output. In the regularization phase, the discriminator q_φ and the encoder q_ϕ are sequentially updated by minimizing two additional losses, L_2 and L_3 ; both losses are defined as

$$L_2 = \alpha_2 \cdot f_B(q_\varphi(\mathbf{h}, \mathbf{a}^{\text{emb}}, s), \mathbf{0}) + (1 - \alpha_2) \cdot f_B(q_\varphi(\mathbf{h}', \mathbf{a}^{\text{emb}}, s), \mathbf{1}), \quad (17)$$

$$L_3 = f_B(q_\varphi(\mathbf{h}, \mathbf{a}^{\text{emb}}, s), \mathbf{1}), \quad (18)$$

where function f_B calculates the binary cross entropy between two inputs, and α_2 is also a positive scalar less than one. As shown in Fig. 6(a), the discriminator q_e will output a vector of 1 when it is provided with $\mathbf{h}'$, a variable from the prior distribution. We designate a standard normal distribution $N(0, 1)$ as the prior to generate $\mathbf{h}'$. The prior distribution can be arbitrary [66]. A vector of $\mathbf{0}$ will be output if the encoder's output $\mathbf{h}$ is fed into the discriminator. Therefore, minimizing loss L_2 can train the discriminator's ability to recognize latent variables output by the encoder. Furthermore, minimizing loss L_3 can force the encoder to generate latent variables with the expected distribution. Specifically, the discriminator's output is fixed at 1 for comparison in the binary cross entropy, and during backpropagation, only the encoder's parameters are updated. In this adversarial way, the encoder's outputs (i.e., latent variables $\mathbf{h}$) can spread over the designated prior distribution.

Actually, the pre-training process of our action decoder is semi-supervised due to the inclusion of $(\mathbf{a}^{\text{emb}}, s)$ as part of the input in (17) and (18). Here $(\mathbf{a}^{\text{emb}}, s)$ serves as a label to supervise the action decoder training. Although this label is only fed into the discriminator, it can still influence how the encoder generates $\mathbf{h}$. The use of such labels in the action decoder training enables the embedding table to learn from the AAE, facilitating the mutual learning. In the context of drone-charger control, the action decoder can, during inference, effectively interpret its continuous output a^{con} as the time for observing or recharging based on the decision on a^{dis} . This reflects that our action decoder is able to learn from experiences about how to encourage the drone and charger to achieve higher rewards through collaborations.

3) *Training the Latent Policy Network*: After pre-training the action decoder, we train the TD3-based latent policy network (i.e., the left component of Fig. 4) to generate decodable latent actions. We introduce slight modifications to the original TD3's model architecture, as shown in Fig. 7. Actually, we add an extra linear output head into both actor μ_θ and target actor μ_α , enabling each actor to generate two continuous vectors (i.e., $\mathbf{z}$ and $\mathbf{x}$). Both vectors will be translated by the pre-trained action decoder into the respective original actions, a^{dis} and a^{con} . After the interaction with the environment, the state is updated from s to s' and a reward of r is acquired. Then, a tuple $\langle s, \mathbf{z}, \mathbf{x}, r, s' \rangle$ is put into the experience replay buffer $\mathcal{B}_\mu$. In line 7 of Algorithm 1, the procedure is repeated b_μ times, putting all b_μ

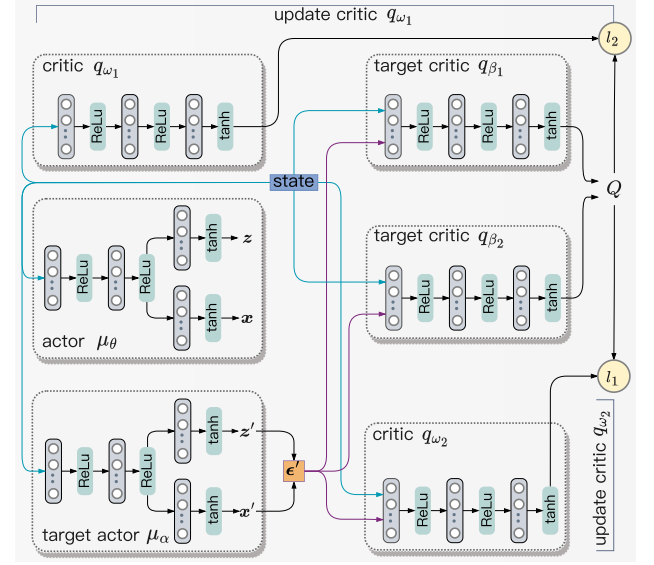


Fig. 7. Implementation of TD3 as HaDMC's latent policy learner, where each linear neural layer includes 256 neurons.

tuples into $\mathcal{B}_\mu$. We use the training algorithm for TD3 [56] to train our latent policy network in line 14 of Algorithm 1.

At the beginning of each iteration (line 8 of Algorithm 1), a mini-batch of b_μ tuples is sampled from the replay buffer $\mathcal{B}_\mu$. For a tuple $\langle s, \mathbf{z}, \mathbf{x}, r, s' \rangle$, state s' is fed into the target actor μ_α , which outputs an action $(\mathbf{z}', \mathbf{x}')$ plus a policy noise ϵ' from a clipped Gaussian distribution $N(0, \sigma')$. After concurrently inputting $(s', \mathbf{z}', \mathbf{x}')$ into the two target critics (i.e., q_{β_1} and q_{β_2}), they output two Q-values. We use the smaller one to calculate the target value y by

$$y = r + \lambda \cdot \min \{q_{\beta_i}(s', \mathbf{z}', \mathbf{x}') | i \in \{1, 2\}\}, \quad (19)$$

where λ is the discount ratio. We calculate the TD error between y and $q_{w_1}(s, \mathbf{z}, \mathbf{x})$, $q_{w_2}(s, \mathbf{z}, \mathbf{x})$ over the current mini-batch. Then, with a learning rate η , we use this error to update parameters of the two critics into ω_1^{new} and ω_2^{new} , respectively. In the while-loop of training the policy network, every 30 iterations (the scheme of delayed policy update), we update the actor μ_θ by one step of gradient ascent with learning rate η , and then update the target actor μ_α and the two target critics q_{β_1} and q_{β_2} , by a soft update scheme: $\alpha^{\text{new}} = \delta \theta^{\text{new}} + (1 - \delta) \alpha$, $\beta_1^{\text{new}} = \delta \omega_1^{\text{new}} + (1 - \delta) \beta_1$, and $\beta_2^{\text{new}} = \delta \omega_2^{\text{new}} + (1 - \delta) \beta_2$. Here the soft-update coefficient δ is set to 5×10^{-3} in our implementation.

VI. EVALUATION

We program a simulator in Python 3.8 and conduct extensive numerical experiments to evaluate the performance of HaDMC, which is trained on PyTorch 2.0.1. The computing environment is Windows 11 Pro and equipped with an NVIDIA RTX 4070ti GPU and an Intel Core i5-13600KF@3.50 GHZ CPU. Our experimental codes are available on GitHub [71].

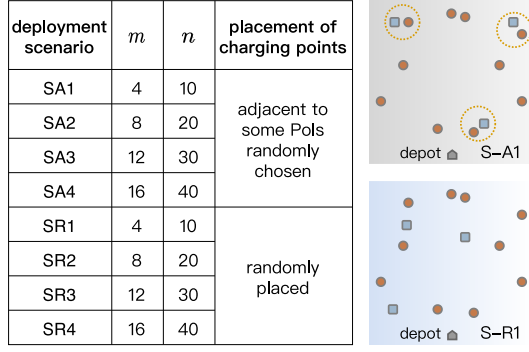


Fig. 8. Scenarios of system deployment involving n PoIs and m charging points (including the depot), and illustrations of two deployment scenarios. Here, the circles and rounded squares represent PoIs and charging points, respectively.

TABLE II
PARAMETERS FOR SYSTEM DEPLOYMENT

parameter	value(s)	parameter	value(s)
drone speed	25	charger speed	10
n	{10, 20, 30, 40}	m	{4, 8, 12, 16}
e	60	γ_o	1
γ_f	1	γ_c	6
$\tau_i^{\min}$	4	$\tau_i^{\max}$	{6, 7, 8}

TABLE III
PARAMETERS FOR MODEL TRAINING

parameter		value	parameter		value
in Algo. 1	$\mathcal{B}_\pi$	1×10^5	in Algo. 1	b_π	1024
	$\mathcal{B}_\mu$	1×10^4		b_μ	256
	n_π	2×10^5		n_μ	8×10^6
	σ	0.1		σ'	0.4
	η	4×10^{-5}		λ	0.995
in loss (16)	α_1	0.5	in loss (17)	α_2	0.5
in rewards (4),(9b)	ξ_1	3	in rewards (12),(13)	ξ_3	-20
	ξ_2	0.2		ξ_4	40

A. Experimental Setup

1) *Setup for System Deployment and Model Training:* In all experiments, the PoIs and charging points are within a square of 1000×1000 . We consider two types of deployment scenarios, denoted by Type-A and Type-R, respectively. As listed in Fig. 8, in Type-A scenarios (SA1 to SA4), the PoIs are randomly placed but maintaining a certain distance from one another, and charging points are located in close to the PoIs. In Type-R scenarios (SR1 to SR4), all charging points are randomly placed, independent of the PoIs.

In each experiment, the drone flies over all PoIs in a clockwise direction, starting from and returning to the depot. This is equivalent to specify the sequence in which the drone visits each PoI. Other parameters related to system deployment are given in Table II. The drone and mobile charger keep their speeds constant while in motion, and the drone's energy and time in landing are ignored in experiments. Parameter settings of our model are shown in Table III. In every 20,000 epochs, we evaluate the training model by running frozen model 50 times in experiments.

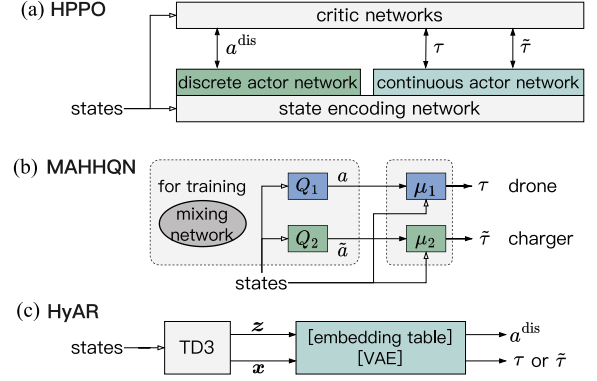


Fig. 9. Implementation and inference processes of baseline models in experiments. Here, the illustration of the three models' training processes is omitted.

2) *Baseline Algorithms:* We compare our HaDMC with two hybrid-action models (HPPO [8] and HyAR [10]), a multi-agent hybrid-action model (MAHHQN [9]), and a greedy algorithm (denoted by GRD). The original HPPO, HyAR and MAHHQN cannot be directly applied to our scenario, and therefore, we made slight modifications to them. The implementation of the three baseline models and their inference processes are outlined in Fig. 9. HPPO is a model that can learn one discrete action and multiple continuous actions. We made its discrete actor output a^{dis} , by which we used (14) to extract the actions of drone and mobile charger, while we drew two continuous outputs (i.e., τ and $\tilde{\tau}$ in Fig. 9(a)) from HPPO's continuous actor to represent drone's observation time and charging duration. Similar to the setup made to HPPO, we initiated the HyAR model but retained only one continuous original action output. This output can be determined based on the value of a^{dis} , as done by our HaDMC. Since MAHHQN is a hybrid-action learner for multi-agent scenarios, as shown in Fig. 9(b), the drone and charger are regarded as two agents, and their actions are learned through two separate pipelines that are jointly trained by a mixing network. Each pipeline first generates a discrete original action and then, a continuous original action. When implementing the three baseline models, we modified their action dimensions and Q-value to ensure consistency with those of our model.

The baseline GRD can find a feasible drone-charger schedule, if it exists, with a greedy policy, without requiring any learning models. Under GRD, the drone always tries to fly to the subsequent PoI p_k and conduct observation for $\tau_k^{\max}$ time. If the drone's remaining energy is not enough for this, it will fly to the charging point closest to p_k , where the charger will fully recharge the drone. If both of the above conditions cannot be met, the drone will fly to the nearest charging point, c_k , from its current location, and the charger needs to stay at or move to c_k . However, if the drone's remaining energy is insufficient to sustain flight to any PoIs or charging points, GRD terminates. In summary, GRD always directs the drone to perform observation tasks and only recharges the drone when absolutely necessary. Additionally, the drone always uses the maximum time for each observation to obtain the highest possible observation utility. We also conduct ablation experiments to investigate the contribution of the critical parts of our action decoder to the overall model.

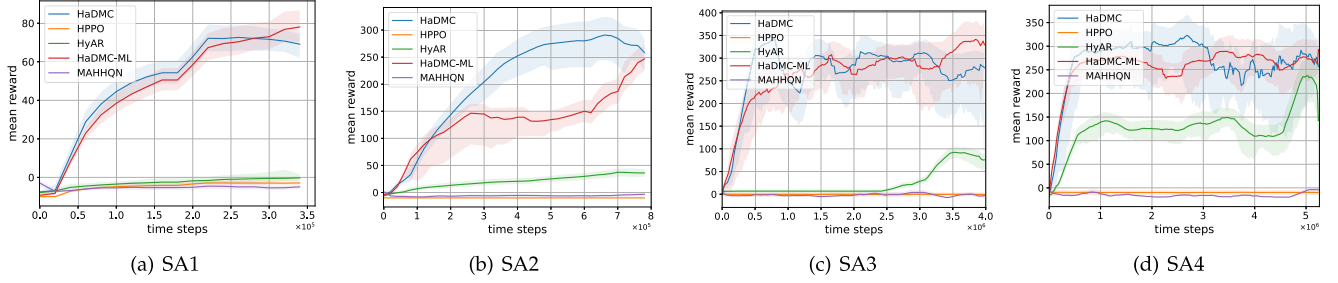


Fig. 10. Reward curves during training for scenarios with charging points close to PoIs.

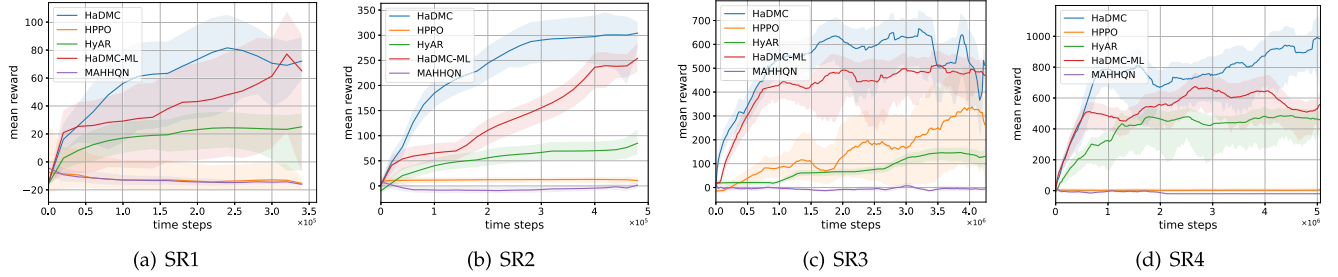


Fig. 11. Reward curves during training for scenarios with charging points randomly deployed.

We remove the mutual learning from the AAE pipeline, keeping other parts of HaDMC unchanged, and this ablation model is denoted as HaDMC-ML.

B. Result Analysis

For a specific type of deployment scenario, we randomly generate 100 deployments and train models on each one. Subsequently, the trained models are executed on additional 50 random deployments of the same type, and the resulting averages are reported to evaluate the performance of these models. Next we will compare our model with baseline algorithms in all eight types of deployment.

Convergence in learning: Figs. 10 and 11 show the learning curves of these models under different deployment scenarios. HaDMC and the ablation model all use an embedding table to convert a high-dimensional continuous latent vector into a discrete action. HPPO does not show convergence in most experiments, although it is originally designed for hybrid-action cases. This is mainly because HPPO generates discrete and continuous actions individually, ignoring the potential correlation between hybrid actions. Moreover, HPPO operates as an indeterministic policy model, where actions are generated through sampling, thereby adding instability to the model training of the model. A surprising discovery is that HyAR exhibits poor convergence in almost all experiments. HyAR only can learn a model when 40 PoIs are involved, as shown in Figs. 10(d) and 11(d). Nevertheless, its learning performance is still lower than our HaDMC and HaDMC-ML. HyAR's hybrid actions are performed by a single agent, while our system requires the participation of two agents, each taking hybrid actions. HyAR cannot effectively learn the cooperative relationship between

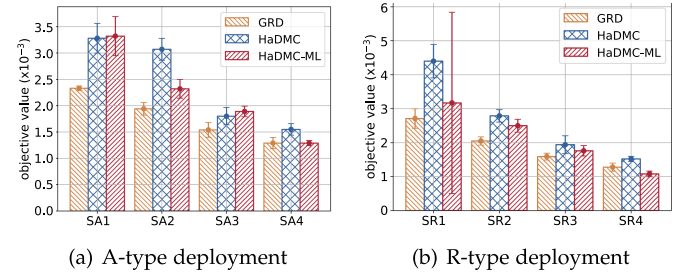


Fig. 12. Comparison of three algorithms in objective value under different deployment scenarios.

the drone and the mobile charger. The MAHHQN model, although designed for multi-agent hybrid-action learning, fails to converge in all eight scenarios; specifically, it does not learn any effective policies at all. Each agent of MAHHQN first generates a discrete action, followed by a continuous action. This sequential approach to action generation during training hinders MAHHQN to effectively learn the relationship between the two actions performed by the same agent. Moreover, even with a high-level mixing network for multi-agent training, MAHHQN cannot fundamentally emphasize the implicit interplay between the drone and mobile charger. For instance, if the drone flies to the next PoI for observation, the mobile charger's decision to charge the drone becomes irrelevant.

Objective values: Since MAHHQN, HPPO and HyAR cannot effectively achieve a reinforcement learning policy, we will only examine HaDMC, GRD, and the ablation model in terms of the objective value (i.e., total observational benefit) and the total time taken upon task completion. Fig. 12 shows that in most deployment scenarios, HaDMC and its ablation model outperform GRD by achieving higher observation efficiency. The ablation

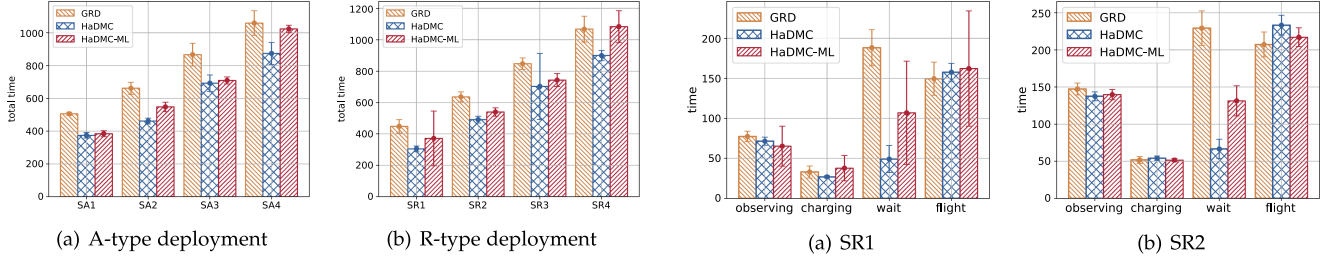


Fig. 13. Comparison of three algorithms in task-completion time under different deployment scenarios.

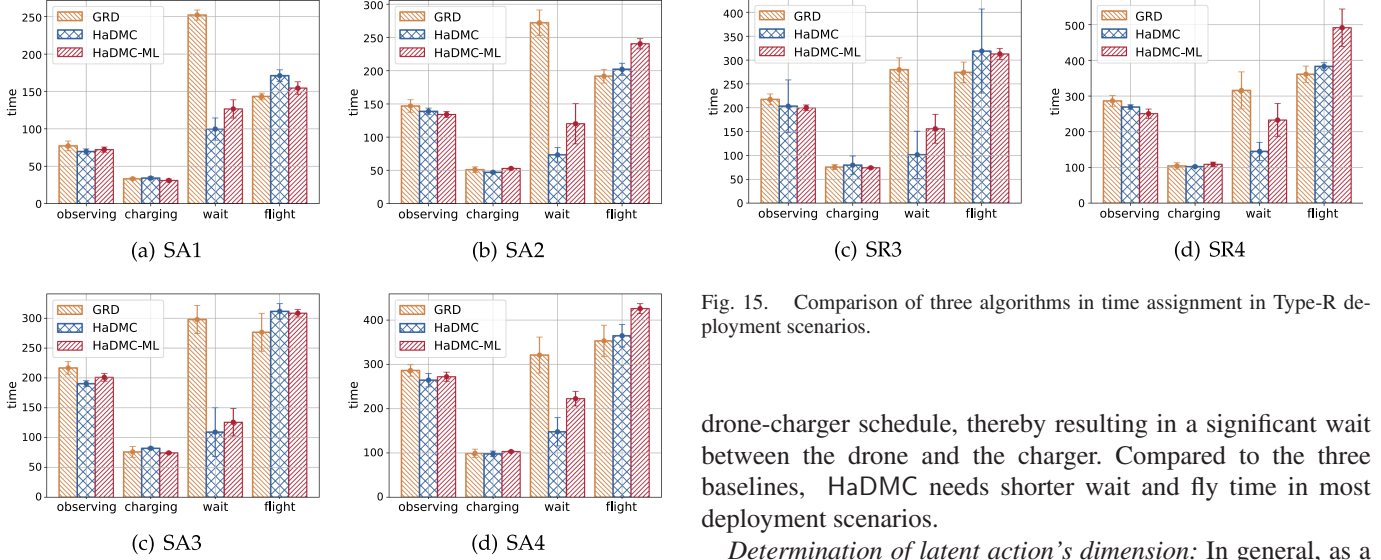


Fig. 14. Comparison of three algorithms in time assignment in Type-A deployment scenarios.

model HaDMC-ML and HaDMC perform similarly only in scenarios of SA1 and SA3. In A-type deployment scenarios, the charging points are adjacent to PoIs, which increases the probability of the drone encountering the charger during flight, thus facilitating their rendezvous with each other. In R-type deployments, however, it can be seen in Fig. 12(b) that the mutual learning contributes to the advantage of HaDMC over HaDMC-ML.

Completion time of tasks: In Fig. 13 we compare the three algorithms in task completion time. GRD almost always takes the longest time to complete tasks, while HaDMC can complete tasks within the shortest time in most scenarios. Noticeably, the ablation model consumes longer time to complete tasks in some scenarios. To understand the behaviors of the three models, we plot their time assignment during performing tasks in Figs. 14 and 15. The time consumed can be divided into four parts: (*observing*) the time of drone observing PoIs, (*charging*) the time in drone charging, (*wait*) the time of the drone or the charger waiting for each other at charging points, and (*flight*) the time of drone in flight. Since GRD is designed to spend the longest possible time in PoI observation, it always consumes the longest observing time during task execution in all experiments. However, GRD only takes current optimal choices for the drone, without considering the potential requirement for cooperative

Fig. 15. Comparison of three algorithms in time assignment in Type-R deployment scenarios.

drone-charger schedule, thereby resulting in a significant wait between the drone and the charger. Compared to the three baselines, HaDMC needs shorter wait and fly time in most deployment scenarios.

Determination of latent action's dimension: In general, as a vector's dimension increases, its capacity for expressing information or representing original action becomes more robust. In other words, for low-dimensional vectors, even if their values are different, they are likely to be mapped to the same output. However, high-dimensional vector will result in increased computational cost. We investigate the effect of latent actions' dimensions on the representation performance, in order to empirically find desirable setup for κ_1 and κ_2 for our model. Recall that κ_1 and κ_2 are the dimensions of the two latent continuous vectors z and x , respectively (see Fig. 4). We let κ_1 and κ_2 be integers ranging from 1 to 19, and examine all possible pairs of them. For a given pair of κ_1 and κ_2 and the corresponding trained model, we generate an additional set of 10,000 distinct latent vectors z and x in uniform distribution, to examine the performance of our action decoder. Each element of these vectors is a random float number within $[-1, 1]$, with four decimal places reserved. We use the action decoder to decode all pairs of vectors, obtaining 10,000 outputs, and then calculate the variance of the occurrence frequency of each output. We prefer to the setups of κ_1 and κ_2 that minimize the variance; in other words, such setups can make the model effectively discern subtle variations in input data that result in distinct outputs. Fig. 16 shows the evaluation of κ_1 and κ_2 when our models are trained under the SA4 and SR4 scenarios. In Fig. 16(a), the effect of latent vectors's dimensions is significant on the variance of outputs. When κ_2 and κ_1 are greater than 5 and 9, respectively, the variances of the embedding table's outputs tend to be zero. Fig. 16(b) shows that although the AAE module is not as sensitive to latent vectors' dimensions

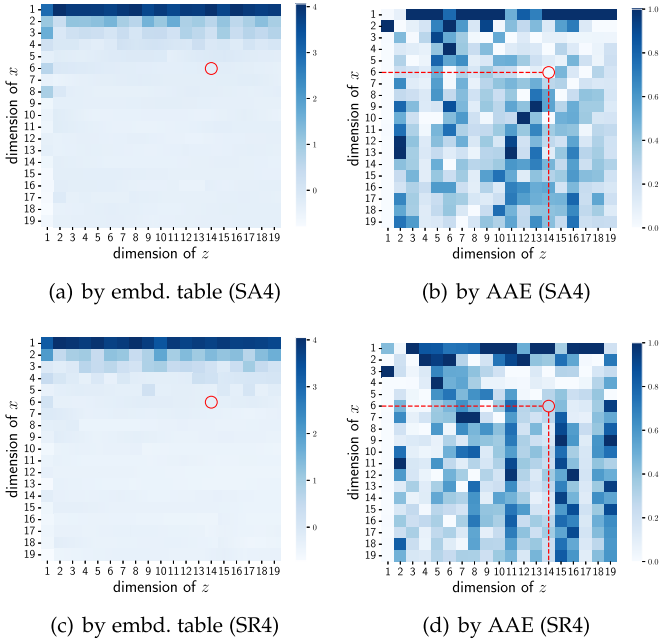


Fig. 16. Representation effectiveness evaluation with different dimensions of latent vectors in the SA4 and SR4 scenarios. In each heatmap matrix, the values (colors) indicate the variance of the occurrence frequency of each output by the two decoding modules of HaDMC.

as the embedding table, higher dimensions are preferable. In Fig. 16(c) and (d), similar results are also found in experiments under the SR4 scenarios. We then empirically set κ_1 and κ_2 to 6 and 14, respectively, in the models for the SA4 and SR3 scenarios. For the scenarios with other scales, the above testing method can also be used to determine appropriate setups of latent vectors' dimensions.

VII. DISCUSSION

The major contribution of our study is address computational challenges of optimizing the observational efficiency of an energy-budgeted drone through a strategic drone-charger scheduling. The challenges arise from the need to make continuous decisions on time and to orchestrate the intricate interplay between discrete and continuous decision-making processes. The success of HaDMC is fundamentally rooted in its ability to learn the interdependency between the drone and mobile charger from hybrid action spaces. Drawing inspiration from the HyAR model, HaDMC integrates the latent-space learning strategy. HyAR is constrained by its generation of hybrid actions for a single agent, but HaDMC transcends the limitation. Unlike MAHHQN, which sequentially generates discrete and continuous actions for each agent, HaDMC recognizes the distinct roles of the drone and mobile charger, both operating in heterogeneous action spaces and requiring a nuanced tailored approach for making collaborative decisions.

The heart of HaDMC is an innovative action decoder, a symphony of an embedding table and an adversarial autoencoder that generate discrete and continuous actions, respectively. Without requiring prior knowledge of the distribution of latent actions, our action decoder is trained with a mutual learning scheme

to exquisitely capture intricate interdependencies between the drone and mobile charger, and it simultaneously generates discrete and continuous actions to emphasize the coordination among hybrid actions. Additionally, the action decoder learns a combination of discrete actions for both drone and mobile charger, rather than learning their discrete actions separately. This combination strategy fortifies the model's ability to internalize the nuanced inter-agent dynamics in discrete spaces during training, as it can be viewed as an encoded representation for these actions. The HaDMC model works with a structured reward function, which is instrumental in promoting exploration and cooperation between drone and mobile charger. Besides, our reward function incorporates a look-ahead policy, not only encouraging drone's exploration but also considering potential charging prospects in future stages. HaDMC features a straightforward architecture. The executable image of trained HaDMC averages 1.33 MB across all the eight experimental scenarios. The training process takes 55 seconds for every 10^4 steps on average, coupled with an inference time measured in milliseconds, making it well-suited for deployment in embedded computing environments.

HaDMC is designed to solve a computationally intractable optimization problem through reinforcement learning. A limitation of HaDMC is that it does not fully take into account environmental factors, such as weather conditions and obstacles [14], which may influence drone's flight and charger's movement. In other words, HaDMC can perform well in stable environments that support both drone and mobile charger to maintain constant speeds in motion and normal operations during observation or recharging process. Given an application-specific scenario, one feasible way of overcoming this limitation is to incorporate more environmental states into HaDMC and its training. For example, we can consider the velocity of the drone flying to its next destination as an additional action, while the weather conditions affecting the flight speed can be viewed as part of state information. These elements can be easily integrated into our model. Specifically, we can modify the latent policy network of HaDMC such that it can generate a third latent continuous action v regarding flight speed; subsequently, we feed the concatenation of v and z into the action decoder of HaDMC, which, with a dimension adjustment, will then output all original actions. In this way, the drone can determine its destination, flight speed, and operation. When training the slightly-modified HaDMC, it is feasible to use datasets from realistic environments to identify the factors that may influence flight speed.

VIII. CONCLUSION

In this paper, we investigate a drone application scenario involving a mobile charger that can recharge the drone's battery to extend its operational lifespan. We focus on the drone-charger scheduling issue, which entails a multi-stage decision process involving two agents that both generate discrete-continuous hybrid actions. We represent the first attempt to address this particular issue, and pioneer a deep reinforcement learning framework, HaDMC, to learn an effective hybrid-action policy model, by which the drone and mobile charger can take cooperative actions

to find a solution to enhancing the drone's observation efficiency. HaDMC employs the representation learning paradigm, using an action decoder to translate latent actions into original actions for the drone and mobile charger. Our action decoder operates through two separate pipelines to generate hybrid actions, without requiring prior knowledge of the distribution of latent actions. To foster cooperation between hybrid actions, a mutual learning scheme is integrated into the model's design and training. Experimental results show HaDMC's effectiveness and efficiency. We believe that our design may offer insights into addressing scheduling challenges involving multiple agents taking cooperative hybrid actions. The application of HaDMC to scenarios involving multiple drones and chargers warrants further investigation due to larger action spaces and more complex interdependencies among actions. This aspect is earmarked for our future research endeavors.

REFERENCES

- [1] M. Mozaffari, W. Saad, M. Bennis, Y.-H. Nam, and M. Debbah, "A tutorial on UAVs for wireless networks: Applications, challenges, and open problems," *IEEE Commun. Surveys Tut.*, vol. 21, no. 3, pp. 2334–2360, Third Quarter 2019.
- [2] Z. Wei et al., "UAV-assisted data collection for Internet of Things: A survey," *IEEE Internet Things J.*, vol. 9, no. 17, pp. 15460–15483, Sep. 2022.
- [3] Y. Bai, H. Zhao, X. Zhang, Z. Chang, R. Jäntti, and K. Yang, "Toward autonomous multi-UAV wireless network: A survey of reinforcement learning-based approaches," *IEEE Commun. Surveys Tut.*, vol. 25, no. 4, pp. 3038–3067, Fourth Quarter 2023.
- [4] Global commercial drones market size, 2022. [Online]. Available: <https://www.blueweaveconsulting.com/report/commercial-drone-market/report-sample>
- [5] M. N. Boukobrine, Z. Zhou, and M. Benbouzid, "A critical review on unmanned aerial vehicles power supply and energy management: Solutions, strategies, and prospects," *Appl. Energy*, vol. 255, 2019, Art. no. 113823.
- [6] P. K. Chittoor, B. Chokkalingam, and L. Mihet-Popa, "A review on UAV wireless charging: Fundamentals, applications, charging techniques and standards," *IEEE Access*, vol. 9, pp. 69235–69266, 2021.
- [7] H. Dong, Z. Ding, and S. Zhang, Eds., *Deep Reinforcement Learning: Fundamentals, Research and Applications*, 1st ed. Berlin, Germany: Springer, Jun. 2020.
- [8] Z. Fan, R. Su, W. Zhang, and Y. Yu, "Hybrid actor-critic reinforcement learning in parameterized action space," in *Proc. 28th Int. Joint Conf. Artif. Intell.*, 2019, pp. 2279–2285.
- [9] H. Fu, H. Tang, J. Hao, Z. Lei, Y. Chen, and C. Fan, "Deep multi-agent reinforcement learning with discrete-continuous hybrid action spaces," in *Proc. 28th Int. Joint Conf. Artif. Intell.*, 2019, pp. 2329–2335.
- [10] B. Li et al., "HyAR: Addressing discrete-continuous action reinforcement learning via hybrid action representation," in *Proc. Int. Conf. Learn. Representations*, 2022, pp. 1–22.
- [11] X. Xu, H. Zhao, H. Yao, and S. Wang, "A blockchain-enabled energy-efficient data collection system for UAV-assisted IoT," *IEEE Internet Things J.*, vol. 8, no. 4, pp. 2431–2443, Feb. 2021.
- [12] X. Yuan, Y. Hu, J. Zhang, and A. Schmeink, "Joint user scheduling and UAV trajectory design on completion time minimization for UAV-aided data collection," *IEEE Trans. Wireless Commun.*, vol. 22, no. 6, pp. 3884–3898, Jun. 2023.
- [13] X. Ma, M. Huang, W. Ni, M. Yin, J. Min, and A. Jamalipour, "Balancing time and energy efficiency by sizing clusters: A new data collection scheme in UAV-aided large-scale Internet of Things," *IEEE Internet Things J.*, vol. 11, no. 6, pp. 9355–9367, Mar. 2024.
- [14] C. Qu, R. Singh, A. E. Morel, F. B. Sorbelli, P. Calyam, and S. K. Das, "Obstacle-aware and energy-efficient multi-drone coordination and networking for disaster response," in *Proc. 17th Int. Conf. Netw. Service Manage.*, 2021, pp. 446–454.
- [15] M. Sun, X. Xu, X. Qin, and P. Zhang, "AoI-energy-aware UAV-assisted data collection for IoT networks: A deep reinforcement learning method," *IEEE Internet Things J.*, vol. 8, no. 24, pp. 17275–17289, Dec. 2021.
- [16] H. Hu, K. Xiong, G. Qu, Q. Ni, P. Fan, and K. B. Letaief, "AoI-minimal trajectory planning and data collection in UAV-assisted wireless powered IoT networks," *IEEE Internet Things J.*, vol. 8, no. 2, pp. 1211–1223, Jan. 2021.
- [17] K. Li, W. Ni, E. Tovar, and M. Guizani, "Joint flight cruise control and data collection in UAV-aided Internet of Things: An onboard deep reinforcement learning approach," *IEEE Internet Things J.*, vol. 8, no. 12, pp. 9787–9799, Jun. 2021.
- [18] X. Wang, M. C. Gursay, T. Erpek, and Y. E. Sagduyu, "Learning-based UAV path planning for data collection with integrated collision avoidance," *IEEE Internet Things J.*, vol. 9, no. 17, pp. 16663–16676, Sep. 2022.
- [19] L. Wang, K. Wang, C. Pan, W. Xu, N. Aslam, and A. Nallanathan, "Deep reinforcement learning based dynamic trajectory control for UAV-assisted mobile edge computing," *IEEE Trans. Mobile Comput.*, vol. 21, no. 10, pp. 3536–3550, Oct. 2022.
- [20] J. Ji, K. Zhu, and L. Cai, "Trajectory and communication design for cache-enabled UAVs in cellular networks: A deep reinforcement learning approach," *IEEE Trans. Mobile Comput.*, vol. 22, no. 10, pp. 6190–6204, Oct. 2023.
- [21] X. Yang, S. Fu, B. Wu, and M. Zhang, "A survey of key issues in UAV data collection in the Internet of Things," in *Proc. 2020 IEEE Int. Symp. Dependable Auton. Secure Comput.*, 2020, pp. 410–413.
- [22] H. Kurunathan, H. Huang, K. Li, W. Ni, and E. Hossain, "Machine learning-aided operations and communications of unmanned aerial vehicles: A contemporary survey," *IEEE Commun. Surveys Tut.*, vol. 26, no. 1, pp. 496–533, First Quarter 2024.
- [23] M. Li, S. He, and H. Li, "Minimizing mission completion time of UAVs by jointly optimizing the flight and data collection trajectory in UAV-enabled WSNs," *IEEE Internet Things J.*, vol. 9, no. 15, pp. 13498–13510, Aug. 2022.
- [24] H. Menouar, I. Guvenc, K. Akkaya, A. S. Uluagac, A. Kadri, and A. Tuncer, "UAV-enabled intelligent transportation systems for the smart city: Applications and challenges," *IEEE Commun. Mag.*, vol. 55, no. 3, pp. 22–28, Mar. 2017.
- [25] Y. Wang, Z. Su, N. Zhang, and R. Li, "Mobile wireless rechargeable UAV networks: Challenges and solutions," *IEEE Commun. Mag.*, vol. 60, no. 3, pp. 33–39, Mar. 2022.
- [26] C. Smith, "Flying drones could soon re-charge whilst airborne with new technology," Oct. 2016. [Online]. Available: <https://www.imperial.ac.uk/news/175318/flying-drones-could-soon-re-charge-whilst/>
- [27] 2023. [Online]. Available: <https://powermat.com/wireless-charging-technology-for-drones/>
- [28] 2022. [Online]. Available: <https://www.wibotic.com/wp-content/uploads/2021/05/Wibotic-Aerial-Datasheet>
- [29] 2023. [Online]. Available: <https://clearpathrobotics.com/warthog-unmanned-ground-vehicle-robot/>
- [30] J. Yao and N. Ansari, "QoS-aware rechargeable UAV trajectory optimization for sensing service," in *Proc. 2019 IEEE Int. Conf. Commun.*, 2019, pp. 1–6.
- [31] W. Chen, S. Zhao, Q. Shi, and R. Zhang, "Resonant beam charging-powered UAV-assisted sensing data collection," *IEEE Trans. Veh. Technol.*, vol. 69, no. 1, pp. 1086–1090, Jan. 2020.
- [32] N. H. Chu, D. T. Hoang, D. N. Nguyen, N. Van Huynh, and E. Dutkiewicz, "Joint speed control and energy replenishment optimization for UAV-assisted IoT data collection with deep reinforcement transfer learning," *IEEE Internet Things J.*, vol. 10, no. 7, pp. 5778–5793, Apr. 2023.
- [33] S. Fu et al., "Energy-efficient UAV-enabled data collection via wireless charging: A reinforcement learning approach," *IEEE Internet Things J.*, vol. 8, no. 12, pp. 10209–10219, Jun. 2021.
- [34] M. Fan et al., "Deep reinforcement learning for UAV routing in the presence of multiple charging stations," *IEEE Trans. Veh. Technol.*, vol. 72, no. 5, pp. 5732–5746, May 2023.
- [35] L. Zhang, A. Celik, S. Dang, and B. Shihada, "Energy-efficient trajectory optimization for UAV-assisted IoT networks," *IEEE Trans. Mobile Comput.*, vol. 21, no. 12, pp. 4323–4337, Dec. 2022.
- [36] M. Li, L. Liu, Y. Gu, Y. Ding, and L. Wang, "Minimizing energy consumption in wireless rechargeable UAV networks," *IEEE Internet Things J.*, vol. 9, no. 5, pp. 3522–3532, Mar. 2022.
- [37] C. H. Liu, C. Piao, and J. Tang, "Energy-efficient UAV crowdsensing with multiple charging stations by deep learning," in *Proc. IEEE Conf. Comput. Commun.*, 2020, pp. 199–208.
- [38] J. Xu, X. Kang, R. Zhang, Y.-C. Liang, and S. Sun, "Optimization for master-UAV-powered auxiliary-aerial-IRS-assisted IoT networks: An option-based multi-agent hierarchical deep reinforcement learning approach," *IEEE Internet Things J.*, vol. 9, no. 22, pp. 22887–22902, Nov. 2022.

- [39] R. G. Ribeiro, L. P. Cota, T. A. M. Euzébio, J. A. Ramírez, and F. G. Guimarães, "Unmanned-aerial-vehicle routing problem with mobile charging stations for assisting search and rescue missions in postdisaster scenarios," *IEEE Trans. Syst., Man, Cybern.: Syst.*, vol. 52, no. 11, pp. 6682–6696, Nov. 2022.
- [40] N. Liu et al., "Dynamic charging strategy optimization for UAV-assisted wireless rechargeable sensor networks based on deep q-network," *IEEE Internet Things J.*, vol. 11, no. 12, pp. 21125–21134, Jun. 2024.
- [41] N. Mazyavkina, S. Sviridov, S. Ivanov, and E. Burnaev, "Reinforcement learning for combinatorial optimization: A survey," *Comput. Oper. Res.*, vol. 134, 2021, Art. no. 105400.
- [42] K. Arulkumaran, M. P. Deisenroth, M. Brundage, and A. A. Bharath, "Deep reinforcement learning: A brief survey," *IEEE Signal Process. Mag.*, vol. 34, no. 6, pp. 26–38, Nov. 2017.
- [43] X. Wang et al., "Deep reinforcement learning: A survey," *IEEE Trans. Neural Netw. Learn. Syst.*, vol. 35, no. 4, pp. 5064–5078, Apr. 2024.
- [44] J. Kober, J. A. Bagnell, and J. Peters, "Reinforcement learning in robotics: A survey," *Int. J. Robot. Res.*, vol. 32, no. 11, pp. 1238–1274, 2013.
- [45] C. Yu, J. Liu, S. Nemati, and G. Yin, "Reinforcement learning in healthcare: A survey," *ACM Comput. Surv.*, vol. 55, Nov. 2021, Art. no. 5.
- [46] Y. Xiao, J. Liu, J. Wu, and N. Ansari, "Leveraging deep reinforcement learning for traffic engineering: A survey," *IEEE Commun. Surveys Tut.*, vol. 23, no. 4, pp. 2064–2097, Fourth Quarter 2021.
- [47] J. Li, L. Yao, X. Xu, B. Cheng, and J. Ren, "Deep reinforcement learning for pedestrian collision avoidance and human-machine cooperative driving," *Inf. Sci.*, vol. 532, pp. 110–124, 2020.
- [48] V. Mnih et al., "Human-level control through deep reinforcement learning," *Nature*, vol. 518, no. 7540, pp. 529–533, 2015.
- [49] H. V. Hasselt, A. Guez, and D. Silver, "Deep reinforcement learning with double Q-learning," in *Proc. 13th AAAI Conf. Artif. Intell.*, 2016, pp. 2094–2100.
- [50] M. Hessel et al., "Rainbow: Combining improvements in deep reinforcement learning," in *Proc. 32nd AAAI Conf. Artif. Intell.*, 2018, pp. 3215–3222.
- [51] M. Fortunato et al., "Noisy networks for exploration," in *Proc. Int. Conf. Learn. Representations*, 2018, pp. 1–21.
- [52] J. Schulman, S. Levine, P. Moritz, M. Jordan, and P. Abbeel, "Trust region policy optimization," in *Proc. 32nd Int. Conf. Mach. Learn.*, 2015, pp. 1889–1897.
- [53] J. Schulman, F. Wolski, P. Dhariwal, A. Radford, and O. Klimov, "Proximal policy optimization algorithms," 2017, *arXiv: 1707.06347*.
- [54] D. Silver, G. Lever, N. Heess, T. Degris, D. Wierstra, and M. Riedmiller, "Deterministic policy gradient algorithms," in *Proc. 31st Int. Conf. Mach. Learn.*, 2014, pp. 1–387–I–395.
- [55] T. P. Lillicrap et al., "Continuous control with deep reinforcement learning," 2015, *arXiv:1509.02971*.
- [56] S. Fujimoto, H. van Hoof, and D. Meger, "Addressing function approximation error in actor-critic methods," in *Proc. 35th Int. Conf. Mach. Learn.*, 2018, pp. 1587–1596.
- [57] T. Haarnoja, A. Zhou, P. Abbeel, and S. Levine, "Soft actor-critic: Off-policy maximum entropy deep reinforcement learning with a stochastic actor," in *Proc. 35th Int. Conf. Mach. Learn.*, 2018, pp. 1861–1870.
- [58] K. W. Cobbe, J. Hilton, O. Klimov, and J. Schulman, "Phasic policy gradient," in *Proc. 38th Int. Conf. Mach. Learn.*, 2021, pp. 2020–2027.
- [59] J. Xiong et al., "Parametrized deep Q-networks learning: Reinforcement learning with discrete-continuous hybrid action space," 2018, *arXiv: 1810.06394*.
- [60] O. Delalleau, M. Peter, E. Alonso, and A. Logut, "Discrete and continuous action representation for practical RL in video games," in *Proc. Workshop Reinforcement Learn. Games*, 2020, pp. 1–10.
- [61] Y. Gao, Y. Matsunami, S. Miyata, and Y. Akashi, "Multi-agent reinforcement learning dealing with hybrid action spaces: A case study for off-grid oriented renewable building energy system," *Appl. Energy*, vol. 326, 2022, Art. no. 120021.
- [62] T. Wang, X. Fan, K. Cheng, X. Du, H. Cai, and Y. Wang, "Parameterized deep reinforcement learning with hybrid action space for energy efficient data center networks," *Comput. Netw.*, vol. 235, 2023, Art. no. 109989.
- [63] R. J. Williams, "Simple statistical gradient-following algorithms for connectionist reinforcement learning," *Mach. Learn.*, vol. 8, pp. 229–256, 1992.
- [64] M. A. Kramer, "Nonlinear principal component analysis using autoassociative neural networks," *Amer. Inst. Chem. Engineers J.*, vol. 37, no. 2, pp. 233–243, 1991.
- [65] D. P. Kingma and M. Welling, "Auto-encoding variational bayes," 2013, *arXiv:1312.6114*.
- [66] A. Makhzani, J. Shlens, N. Jaitly, I. Goodfellow, and B. Frey, "Adversarial autoencoders," 2015, *arXiv:1511.05644*.
- [67] X. Fan, M. Liu, Y. Chen, S. Sun, Z. Li, and X. Guo, "RIS-assisted UAV for fresh data collection in 3D urban environments: A deep reinforcement learning approach," *IEEE Trans. Veh. Technol.*, vol. 72, no. 1, pp. 632–647, Jan. 2023.
- [68] P.-W. Chou, D. Maturana, and S. Scherer, "Improving stochastic policy gradients in continuous control with deep reinforcement learning using the beta distribution," in *Proc. 34th Int. Conf. Mach. Learn.*, 2017, pp. 834–843.
- [69] I. G. B. Petrazzini and E. A. Antonelo, "Proximal policy optimization with continuous bounded action space via the beta distribution," in *Proc. 2021 IEEE Symp. Ser. Comput. Intell.*, 2021, pp. 1–8.
- [70] K. He, X. Zhang, S. Ren, and J. Sun, "Delving deep into rectifiers: Surpassing human-level performance on ImageNet classification," in *Proc. 2015 IEEE Int. Conf. Comput. Vis.*, 2015, pp. 1026–1034.
- [71] 2024. [Online]. Available: <https://github.com/jizheDou/HaDMC.git>



Jizhe Dou received the BS degree in applied mathematics from the Jiangsu University of Science and Technology, Zhenjiang, China, in 2022. He is currently working toward the master's degree in electronic information engineering with the School of Information and AI, Beijing Forestry University, Beijing, China. His main research interests include reinforcement learning, combinatorial optimization, and wireless algorithm designs.



Haotian Zhang received the BE degree in computer science from the innovation class of Beijing Forestry University, China, in 2022. He is currently working toward the ME degree in computer science with the School of Information and AI, Beijing Forestry University, Beijing, China. His academic interests mainly include combinatorial optimization and wireless algorithm designs, specifically the designs of underlying protocols in IoT.



Yang Luo received the BE degree in Internet-of-Things from the Hubei University of Economics, Wuhan, China, in 2024. She is currently working toward the ME degree in electronic information engineering with the School of Information and AI, Beijing Forestry University, Beijing, China. Her academic interests mainly include embedded networked systems, UAV trajectory planning, wireless networking algorithms, and machine learning.



Guodong Sun (Member, IEEE) is now an associate professor in IoT with the School of Information and AI, Beijing Forestry University, Beijing, China, where he is also the founding director of the Embedded Networked System Lab. Between 2009 and 2012, he worked as a postdoctoral researcher with Tsinghua University, Beijing. From 2016 to 2017, he was a research fellow with the University of North Carolina at Charlotte. He has published more than 20 papers in IEEE/ACM journals and conferences.

His research interests include mobile computing, machine learning, distributed algorithm design, wireless sensor networks, and smart cities.